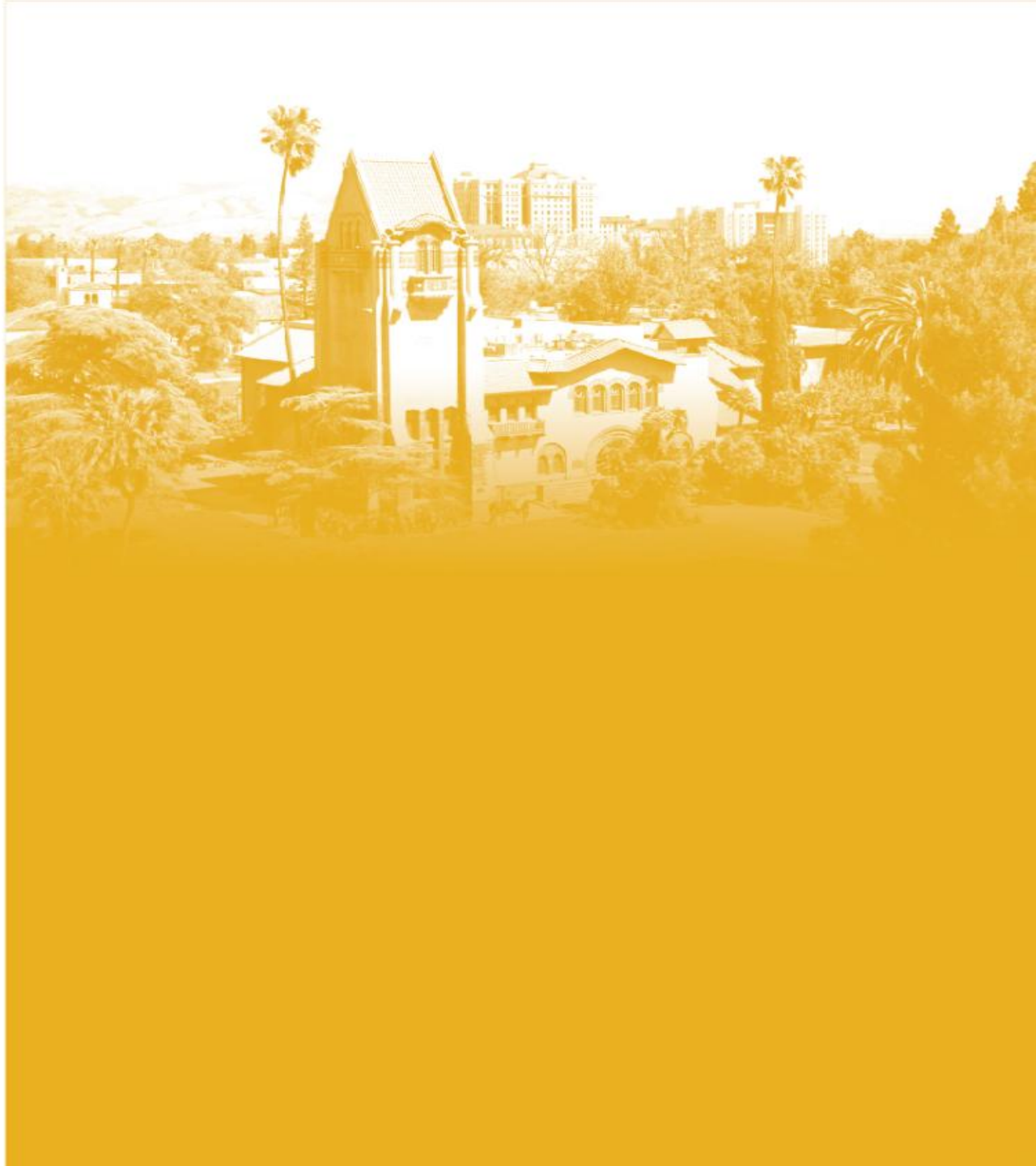




DATA 255
Deep Learning Technologies

Dr. Mohammad Masum



- The human brain contains around 100 billion neurons, with each neuron connecting to approximately 10,000 others.
- Neurons send electrical impulses through axons, which are long, thin structures that branch out extensively.
- At the end of each axon branch, synapses convert electrical impulses into signals that either excite or inhibit the connected neurons, facilitating communication within the brain.

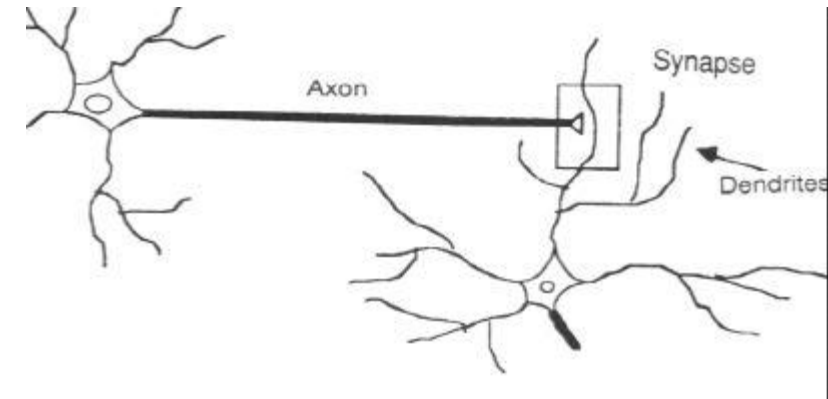
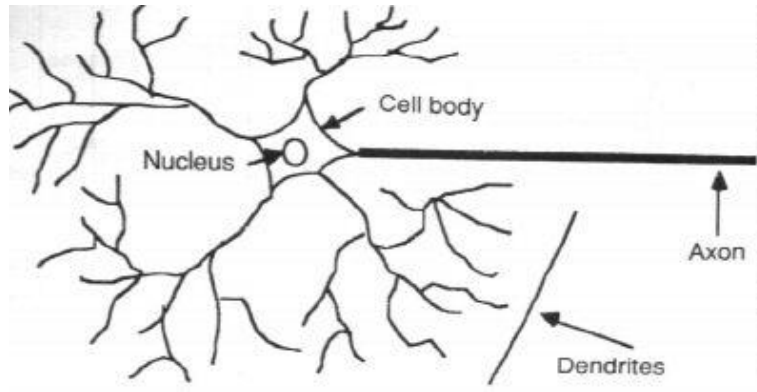
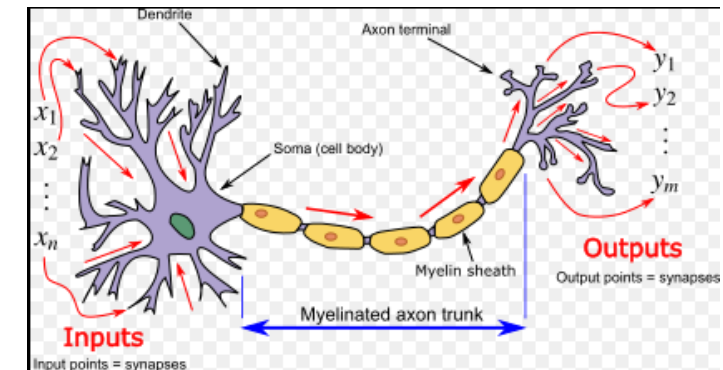
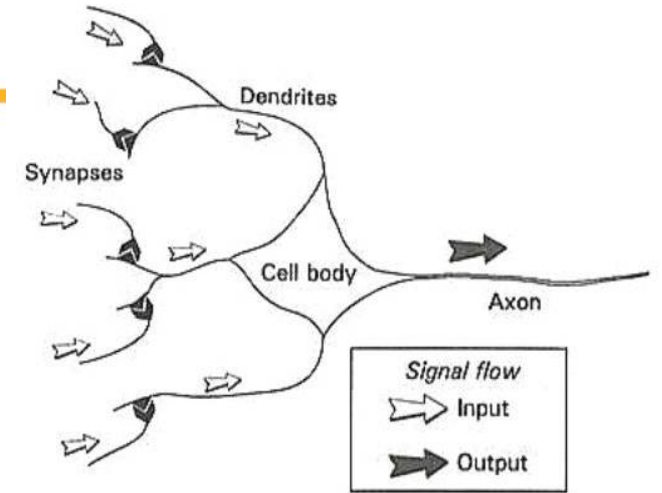


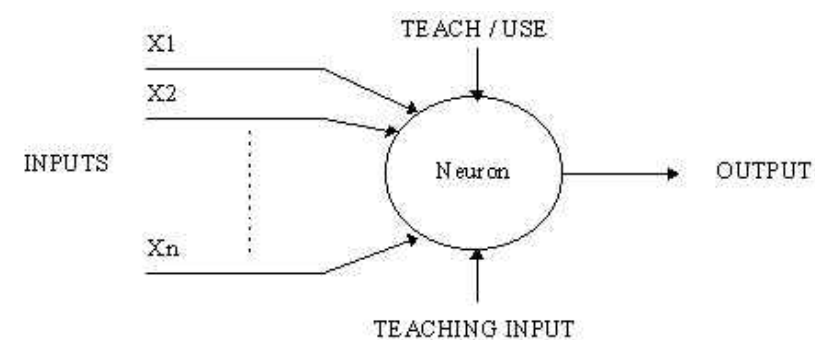
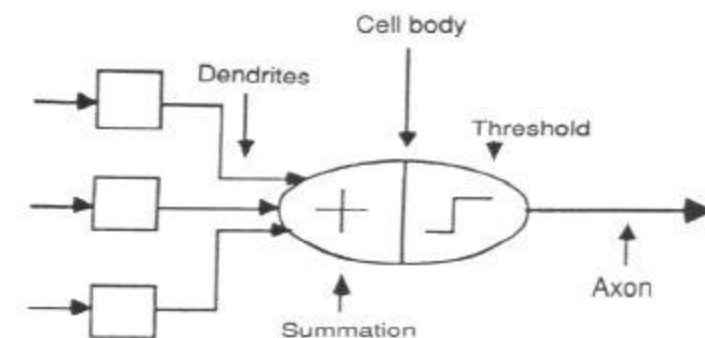
Diagram of a Biological Neuron

- The output of the neuron is an electrical voltage
- This voltage is decided by the cell body, based on the inputs it receives
- This output goes to several other nerve cells, where it acts as an input
- Thus, this single neuron can "contribute" to controlling several other neurons
- We follow this architecture
 - First trying to deduce the essential features of the neurons and their interconnections
 - Program a computer to simulate these features

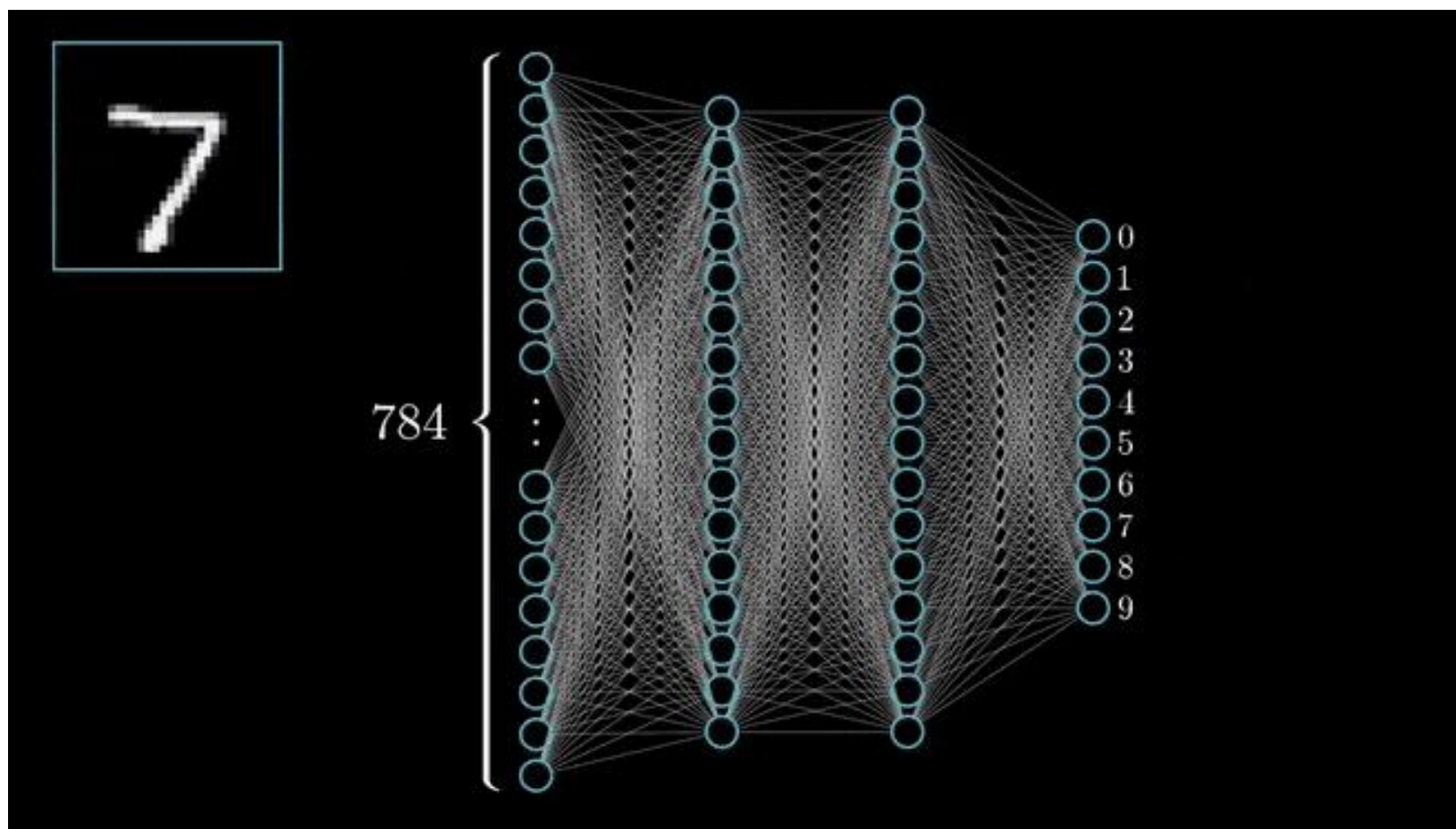


An Artificial Neuron

- An artificial neuron
 - many inputs and one output
 - can be trained to fire/activate (or not), for particular input patterns
- The **firing rule** is an important concept in neural networks and accounts for their high flexibility
 - A firing rule determines whether a neuron should fire for any input pattern

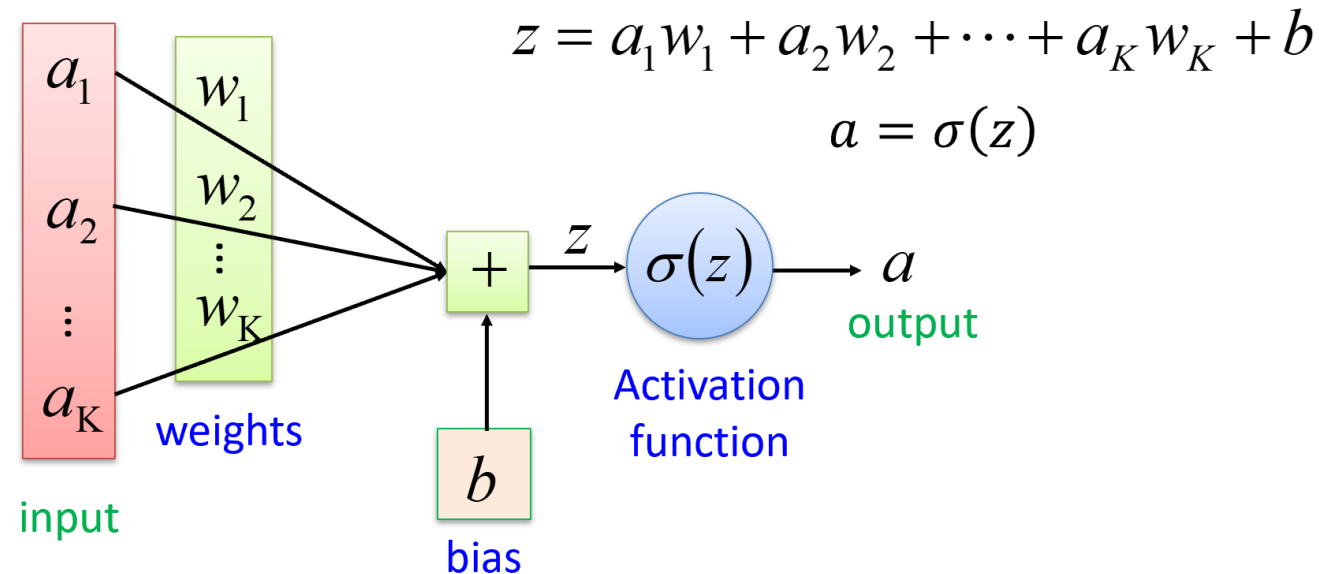


An Artificial Neuron



Elements of Neural Network

- Perceptron is the basic architecture of a neural network
 - A single neuron takes a set of inputs and output a number or $f: R^K \rightarrow R$
 - Why we need to add biases into the network? Hint- linear regression



Elements of Neural Network

```
1 from keras.models import Sequential
2 from keras.layers import Dense
3
4 # Define the model
5 model = Sequential()
6 model.add(Dense(5, input_shape=(10,),
7                 use_bias = False))
8 model.add(Dense(1, use_bias = False))
```

```
1 model.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
=====		
dense_14 (Dense)	(None, 5)	50
dense_15 (Dense)	(None, 1)	5
=====		
Total params: 55 (220.00 Byte)		
Trainable params: 55 (220.00 Byte)		
Non-trainable params: 0 (0.00 Byte)		

```
1 # Define the model
2 model = Sequential()
3 model.add(Dense(5, input_shape=(10,),
4                 use_bias = True))
5 model.add(Dense(1, use_bias = True))
6 model.summary()
```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
=====		
dense_16 (Dense)	(None, 5)	55
dense_17 (Dense)	(None, 1)	6
=====		
Total params: 61 (244.00 Byte)		
Trainable params: 61 (244.00 Byte)		
Non-trainable params: 0 (0.00 Byte)		

Elements of Neural Network

```
1 # Define the model
2 model = Sequential()
3 model.add(Dense(5, input_shape=(20,),
4                 use_bias = True))
5 model.add(Dense(1, use_bias = True))
6 model.summary()
```

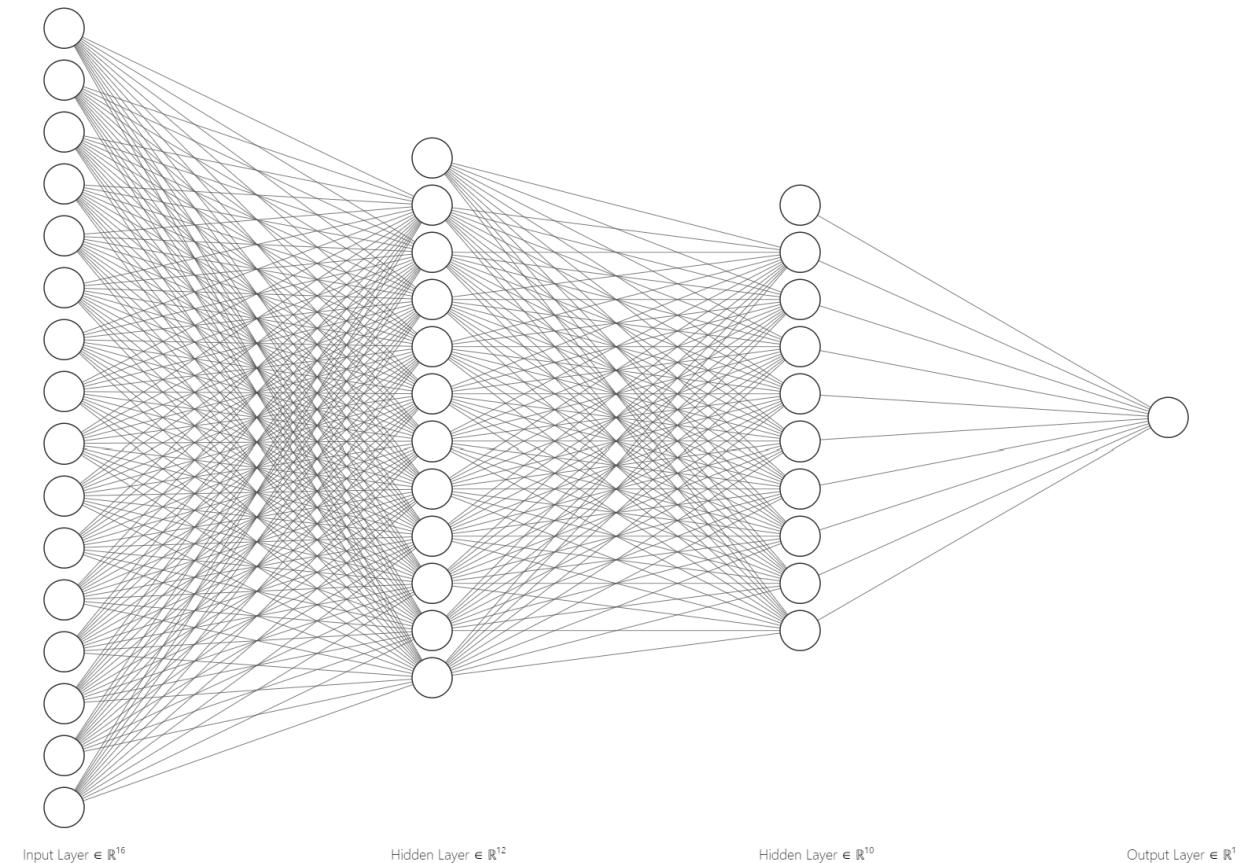
Total learnable parameters=?

```
1 # Define the model
2 model = Sequential()
3 model.add(Dense(5, input_shape=(20,),
4                 use_bias = False))
5 model.add(Dense(1, use_bias = False))
6 model.summary()
```

Total learnable parameters=?

Multi Layer Perceptron

- Deep NNs have many hidden layers
 - **Fully connected** or Dense or Multilayer Perceptron (MLP) – each neuron connected to all neurons in the succeeding layer



ANN –Forward Pass

ANN-Loss function

- What is error?
- What is loss function?
- What is cost function?

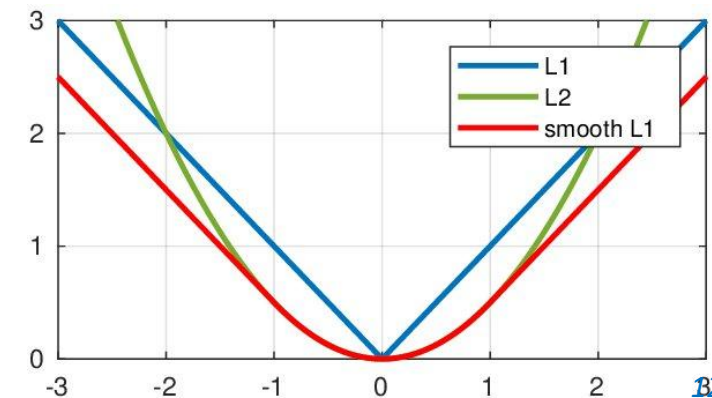
ANN-Loss function

- Mean Squared Error (MSE) or L2 Loss
 - Real number prediction
 - Regression (e.g., house price prediction)
 - MSE becomes high for large loss values and decreases as loss approaches towards zero
 - MSE is sensitive to outliers
 - Half of the MSE another version of MSE
- Mean Absolute Error (MAE) or L1 Loss
 - Less sensitive to outliers
- Huber Loss (Smooth L1 Loss)
 - Combination of L1 and L2 loss

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

$$L_{\delta}(y, \hat{y}_i) = \begin{cases} \frac{1}{2}(\hat{y}_i - y_i)^2, & \text{if } |\hat{y}_i - y_i| < \delta \\ \delta|\hat{y}_i - y_i| - \frac{1}{2}\delta^2, & \text{otherwise} \end{cases}$$



ANN-Loss function

- Popular Loss Functions
 - Binary Cross Entropy (Logistic)
 - Output is probability
 - Mainly used for Binary classification (output 0 or 1)
 - Cross-entropy
 - Output is probability
 - Mainly used for multi-class classification
 - Ground truth should be in one hot vector format

$$L = \frac{1}{n} \sum_{j=1}^n \sum_{i=1}^c -(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

$$L = \frac{1}{n} \sum_{j=1}^n \sum_{i=1}^c -(y_i \log(\hat{y}_i))$$

ANN-Loss Function

- In DL, main goal is to minimize the Loss (cost) by achieving optimal set of learnable parameters

$$W^* = \underset{W}{\operatorname{argmin}} J$$

$$J = \frac{1}{n} \sum_{i=1}^n L(\widehat{y}_i, y_i)$$

$$J = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\widehat{y}_i - y_i)^2$$

Gradient Descent

- It is the backbone of to train deep learning
- It is a **first order optimization** algorithm
- The derivative of **multivariate** (objective) function → “**Gradient**”
- Why we use gradient?
 - To find the direction of the minima
- The sign of the **gradient** tells whether the objective function is increasing/decreasing
 - Positive gradient- function is increasing at that point
 - Negative gradient- function is decreasing at that point
- Why it is important to know the sign of the gradient?
 - As this is a minimization problem, we search of the minima in the negative direction of the gradient

Gradient Descent

- Gradient descent algorithm:

$$x_{new} = x - \alpha f'(x)$$

- Calculate gradient at a randomly selected point in the input space
- Find the direction
- How far to move – step size (learning rate)

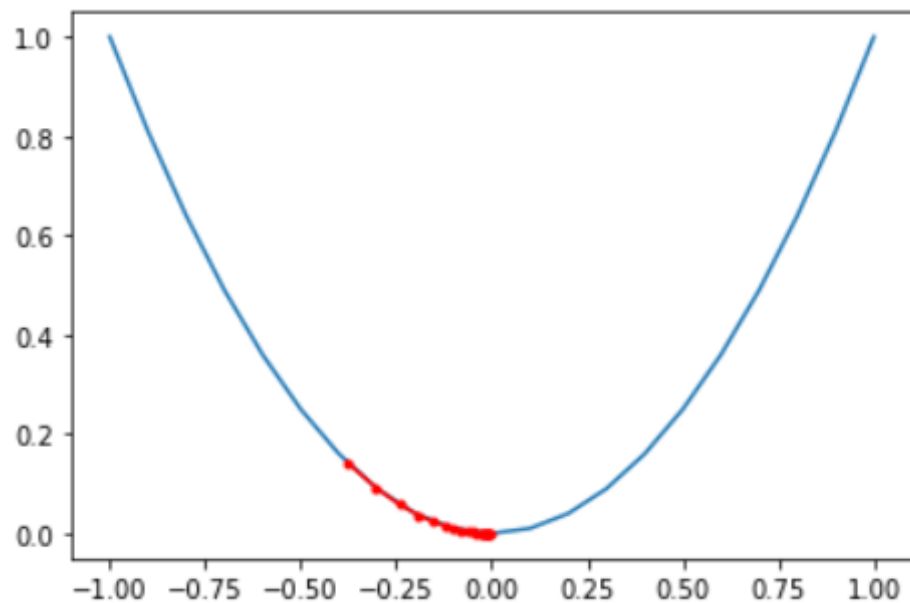
Gradient Descent

- Step size (learning rate)
 - Hyperparameter
 - If step size is too small – slow movement – take a long time to converge
 - If step size is too large – the search may bounce around the search space – may diverge
 - Finding good step size may require some trial and error

Gradient Descent

- Step size (learning rate)

Step size = .1



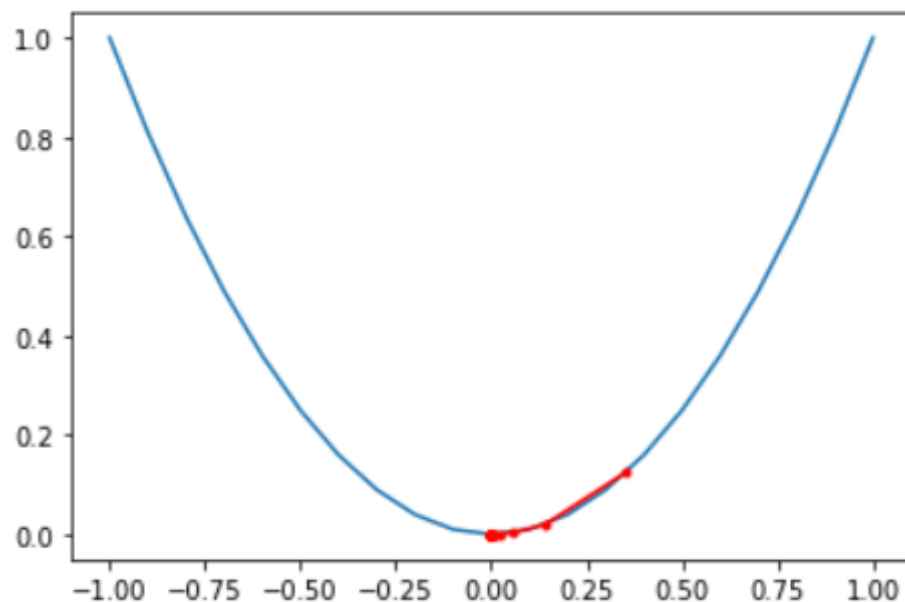
```

>0 f([-0.37803835]) = 0.14291
>1 f([-0.30243068]) = 0.09146
>2 f([-0.24194454]) = 0.05854
>3 f([-0.19355564]) = 0.03746
>4 f([-0.15484451]) = 0.02398
>5 f([-0.12387561]) = 0.01535
>6 f([-0.09910049]) = 0.00982
>7 f([-0.07928039]) = 0.00629
>8 f([-0.06342431]) = 0.00402
>9 f([-0.05073945]) = 0.00257
>10 f([-0.04059156]) = 0.00165
>11 f([-0.03247325]) = 0.00105
>12 f([-0.0259786]) = 0.00067
>13 f([-0.02078288]) = 0.00043
>14 f([-0.0166263]) = 0.00028
>15 f([-0.01330104]) = 0.00018
>16 f([-0.01064083]) = 0.00011
>17 f([-0.00851267]) = 0.00007
>18 f([-0.00681013]) = 0.00005
>19 f([-0.00544811]) = 0.00003
    
```

Gradient Descent

- Step size (learning rate)

Step size = .3

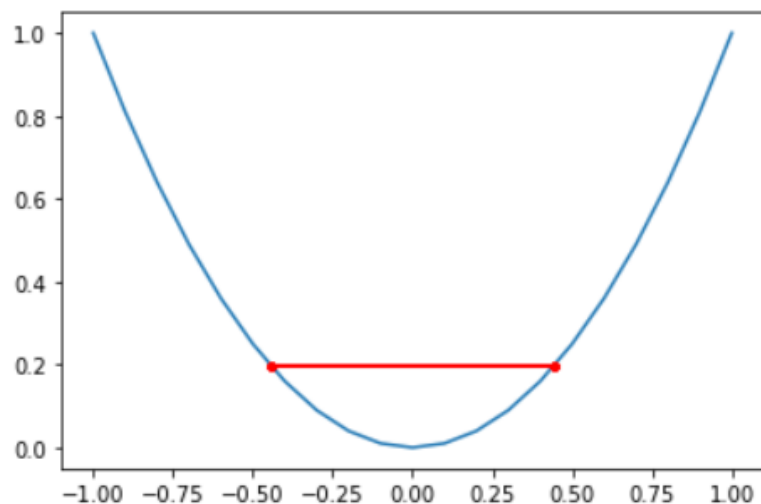


```
>0 f([0.35377041]) = 0.12515
>1 f([0.14150816]) = 0.02002
>2 f([0.05660327]) = 0.00320
>3 f([0.02264131]) = 0.00051
>4 f([0.00905652]) = 0.00008
>5 f([0.00362261]) = 0.00001
>6 f([0.00144904]) = 0.00000
>7 f([0.00057962]) = 0.00000
>8 f([0.00023185]) = 0.00000
>9 f([9.27387908e-05]) = 0.00000
>10 f([3.70955163e-05]) = 0.00000
>11 f([1.48382065e-05]) = 0.00000
>12 f([5.93528261e-06]) = 0.00000
>13 f([2.37411304e-06]) = 0.00000
>14 f([9.49645217e-07]) = 0.00000
>15 f([3.79858087e-07]) = 0.00000
>16 f([1.51943235e-07]) = 0.00000
>17 f([6.07772939e-08]) = 0.00000
>18 f([2.43109176e-08]) = 0.00000
>19 f([9.72436703e-09]) = 0.00000
```

Gradient Descent

- Step size (learning rate)

Step size = 1

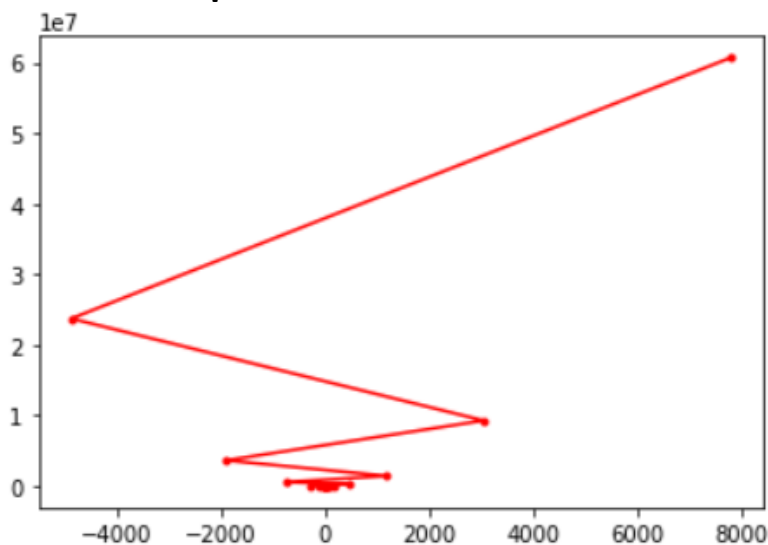


```
>0 f([-0.44198753]) = 0.19535
>1 f([0.44198753]) = 0.19535
>2 f([-0.44198753]) = 0.19535
>3 f([0.44198753]) = 0.19535
>4 f([-0.44198753]) = 0.19535
>5 f([0.44198753]) = 0.19535
>6 f([-0.44198753]) = 0.19535
>7 f([0.44198753]) = 0.19535
>8 f([-0.44198753]) = 0.19535
>9 f([0.44198753]) = 0.19535
>10 f([-0.44198753]) = 0.19535
>11 f([0.44198753]) = 0.19535
>12 f([-0.44198753]) = 0.19535
>13 f([0.44198753]) = 0.19535
>14 f([-0.44198753]) = 0.19535
>15 f([0.44198753]) = 0.19535
>16 f([-0.44198753]) = 0.19535
>17 f([0.44198753]) = 0.19535
>18 f([-0.44198753]) = 0.19535
>19 f([0.44198753]) = 0.19535
```

Gradient Descent

- Step size (learning rate)

Step size = 1.3

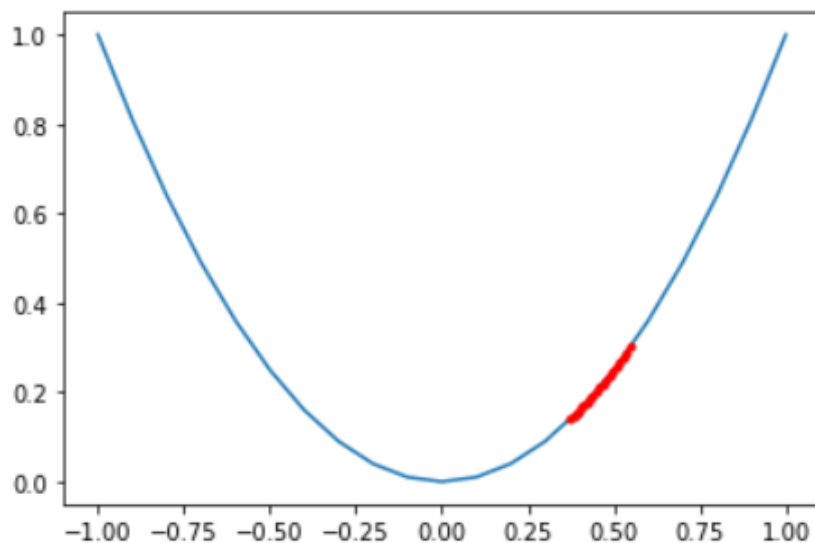


```
>0 f([-1.0314731]) = 1.06394
>1 f([1.65035696]) = 2.72368
>2 f([-2.64057113]) = 6.97262
>3 f([4.22491381]) = 17.84990
>4 f([-6.75986209]) = 45.69574
>5 f([10.81577935]) = 116.98108
>6 f([-17.30524695]) = 299.47157
>7 f([27.68839512]) = 766.64722
>8 f([-44.3014322]) = 1962.61689
>9 f([70.88229152]) = 5024.29925
>10 f([-113.41166643]) = 12862.20608
>11 f([181.45866629]) = 32927.24757
>12 f([-290.33386606]) = 84293.75378
>13 f([464.53418569]) = 215792.00968
>14 f([-743.25469711]) = 552427.54478
>15 f([1189.20751538]) = 1414214.51463
>16 f([-1902.7320246]) = 3620389.15745
>17 f([3044.37123936]) = 9268196.24307
>18 f([-4870.99398298]) = 23726582.38225
>19 f([7793.59037277]) = 60740050.89856
```

Gradient Descent

- Step size (learning rate)

Step size = 0.01 & iteration 20



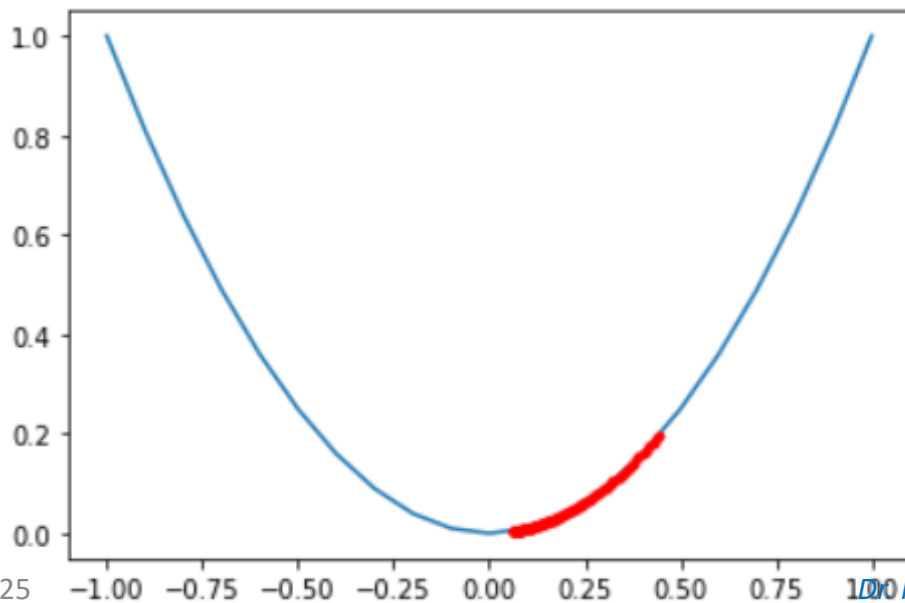
```

>0 f([0.54919471]) = 0.30161
>1 f([0.53821082]) = 0.28967
>2 f([0.5274466]) = 0.27820
>3 f([0.51689767]) = 0.26718
>4 f([0.50655972]) = 0.25660
>5 f([0.49642852]) = 0.24644
>6 f([0.48649995]) = 0.23668
>7 f([0.47676995]) = 0.22731
>8 f([0.46723455]) = 0.21831
>9 f([0.45788986]) = 0.20966
>10 f([0.44873207]) = 0.20136
>11 f([0.43975742]) = 0.19339
>12 f([0.43096228]) = 0.18573
>13 f([0.42234303]) = 0.17837
>14 f([0.41389617]) = 0.17131
>15 f([0.40561825]) = 0.16453
>16 f([0.39750588]) = 0.15801
>17 f([0.38955576]) = 0.15175
>18 f([0.38176465]) = 0.14574
>19 f([0.37412936]) = 0.13997
    
```


Gradient Descent

- Step size (learning rate)

Step size = 0.01 & iteration = 100

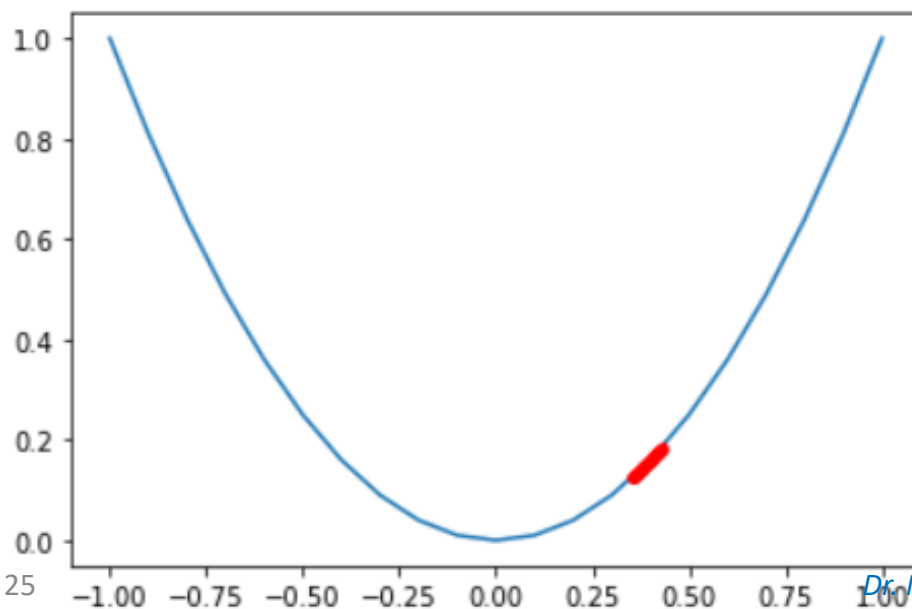


```
>80 f([0.08840912]) = 0.00782
>81 f([0.08664094]) = 0.00751
>82 f([0.08490812]) = 0.00721
>83 f([0.08320996]) = 0.00692
>84 f([0.08154576]) = 0.00665
>85 f([0.07991484]) = 0.00639
>86 f([0.07831655]) = 0.00613
>87 f([0.07675021]) = 0.00589
>88 f([0.07521521]) = 0.00566
>89 f([0.07371091]) = 0.00543
>90 f([0.07223669]) = 0.00522
>91 f([0.07079195]) = 0.00501
>92 f([0.06937611]) = 0.00481
>93 f([0.06798859]) = 0.00462
>94 f([0.06662882]) = 0.00444
>95 f([0.06529624]) = 0.00426
>96 f([0.06399032]) = 0.00409
>97 f([0.06271051]) = 0.00393
>98 f([0.0614563]) = 0.00378
>99 f([0.06022718]) = 0.00363
```


Gradient Descent

- Step size (learning rate)

Step size = 0.001 &
iteration = 100

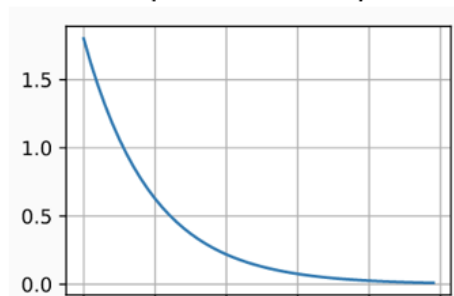


```
>80 f([0.36627158]) = 0.13415
>81 f([0.36553904]) = 0.13362
>82 f([0.36480796]) = 0.13308
>83 f([0.36407835]) = 0.13255
>84 f([0.36335019]) = 0.13202
>85 f([0.36262349]) = 0.13150
>86 f([0.36189824]) = 0.13097
>87 f([0.36117445]) = 0.13045
>88 f([0.3604521]) = 0.12993
>89 f([0.35973119]) = 0.12941
>90 f([0.35901173]) = 0.12889
>91 f([0.35829371]) = 0.12837
>92 f([0.35757712]) = 0.12786
>93 f([0.35686197]) = 0.12735
>94 f([0.35614824]) = 0.12684
>95 f([0.35543595]) = 0.12633
>96 f([0.35472507]) = 0.12583
>97 f([0.35401562]) = 0.12533
>98 f([0.35330759]) = 0.12483
>99 f([0.35260098]) = 0.12433
```

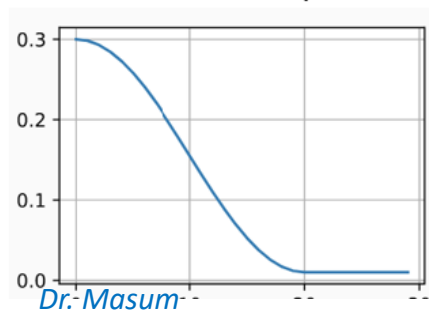
Gradient Descent

- Learning rater scheduling - Change the values of the LR during training
 - Learning rate decay: reducing learning rate over time
 - Approach 1: reduce the LR by some constant systematically
 - E.g., reduce 50% every 10 epochs
 - Approach 2: gradually reduce the learning rate over time
 - E.g., Exponential decay, cosine decay
 - Approach 3: reduce learning rate by a constant whenever validation loss stops improving
 - Warm up: gradually increase the learning rate initially and then cool down until the training

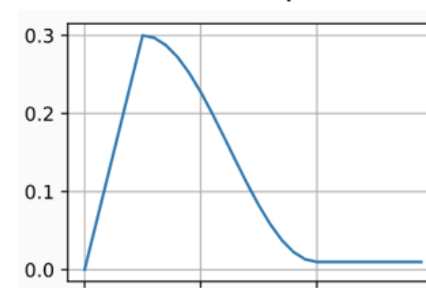
Exponential decay



Cosine decay



Warmup

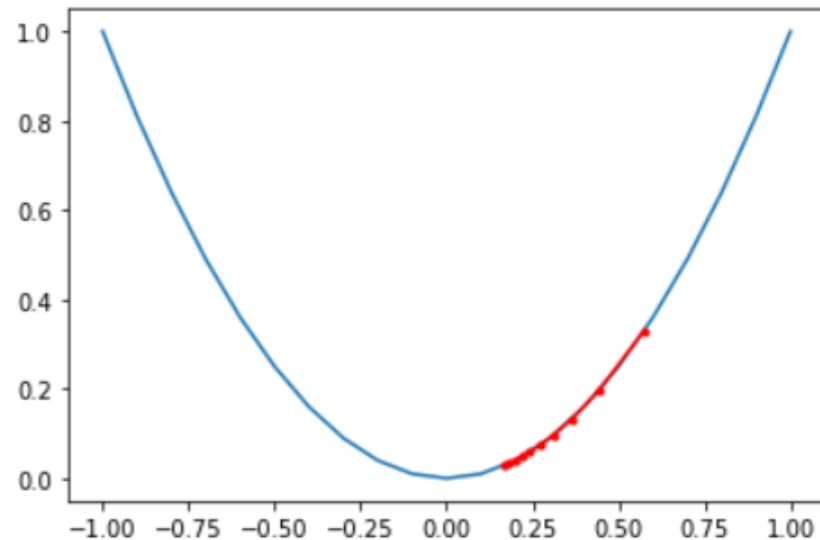


Gradient Descent

- Gradient-based dynamic learning rate
 - If the starting value is far away from the minima $\rightarrow$ the gradient is large
 - Approaches towards minima $\rightarrow$ gradients approaches towards 0
 - Learning rate = learning rate * $\text{abs}(\text{gradient})$

Gradient Descent

- Step size (learning rate) – **Dynamic**
- Gradient-based dynamic learning rate



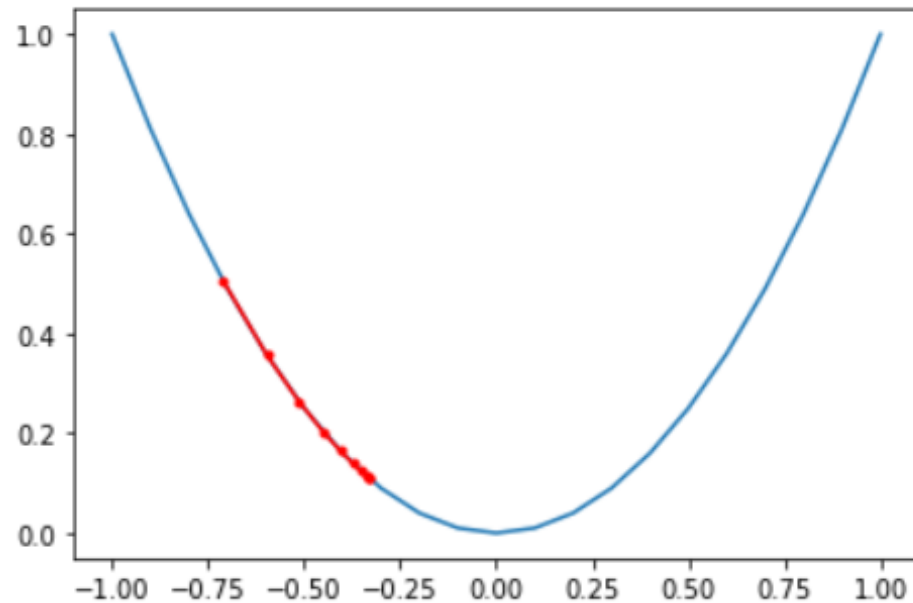
	gradient	step_size
0	[1.7791062749017068]	[0.1779106274901707]
1	[1.1460624474227812]	[0.11460624474227812]
2	[0.8833706207442422]	[0.08833706207442422]
3	[0.7273018900254287]	[0.07273018900254287]
4	[0.6215082821785165]	[0.06215082821785165]
5	[0.5442537732152184]	[0.05442537732152184]
6	[0.48501133928341794]	[0.048501133928341794]
7	[0.43796413943671897]	[0.0437964139436719]
8	[0.3996016219502098]	[0.039960162195020986]
9	[0.3676653306971621]	[0.036766533069716216]

Gradient Descent

- Step size (learning rate) – **Dynamic**
- Time-based dynamic learning rate
 - Adjusting the learning rate according to the training epoch
 - Learning rate = learning rate $(1 - \frac{epoch\ number}{total\ epochs})$

Gradient Descent

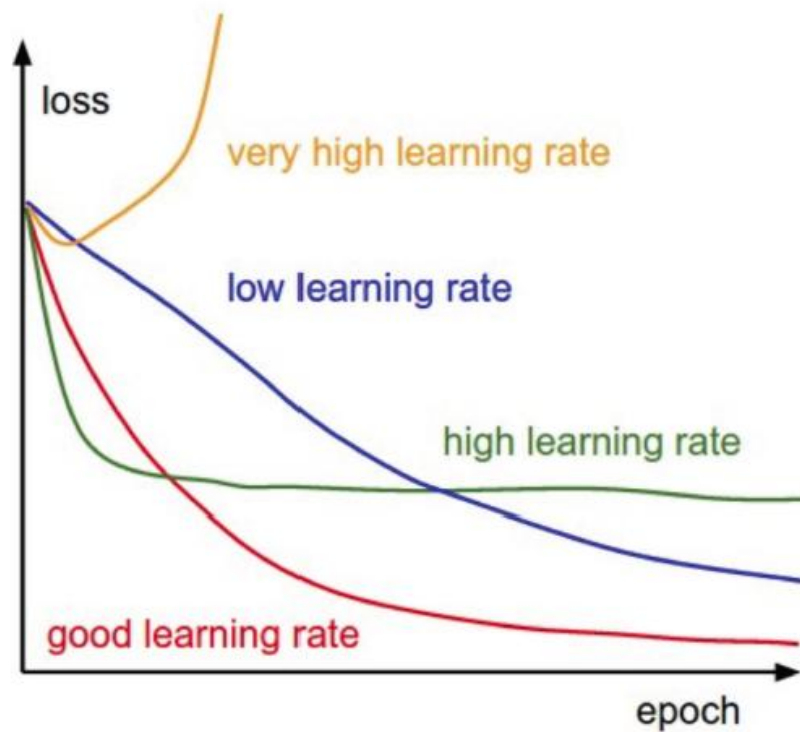
- Step size (learning rate) – **Dynamic**
- Time-based dynamic learning rate



step_size

0	0.10
1	0.09
2	0.08
3	0.07
4	0.06
5	0.05
6	0.04
7	0.03
8	0.02
9	0.01

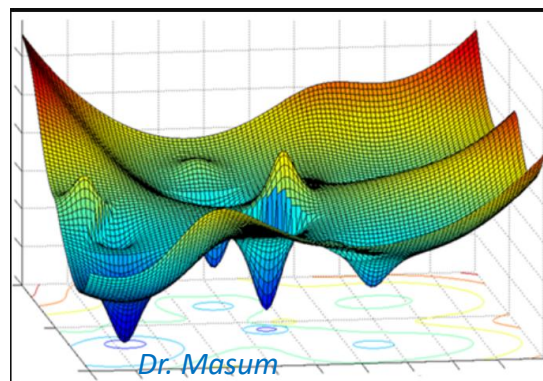
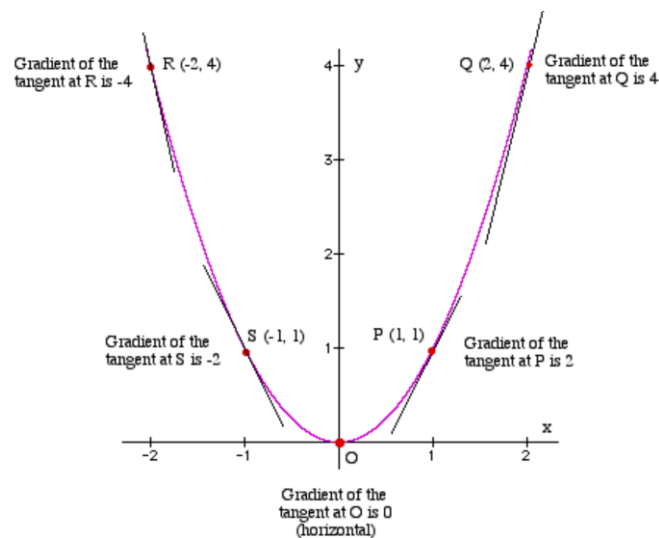
Gradient Descent



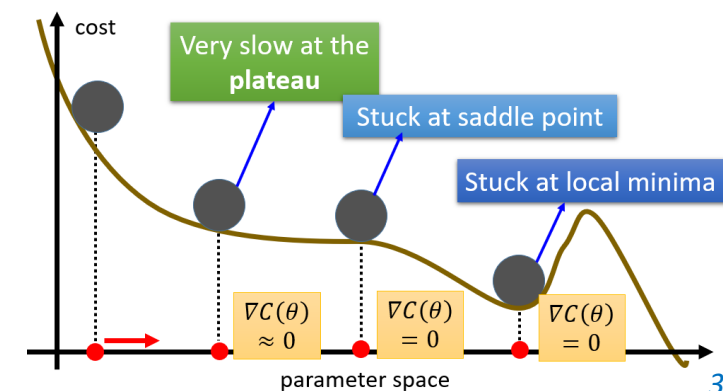
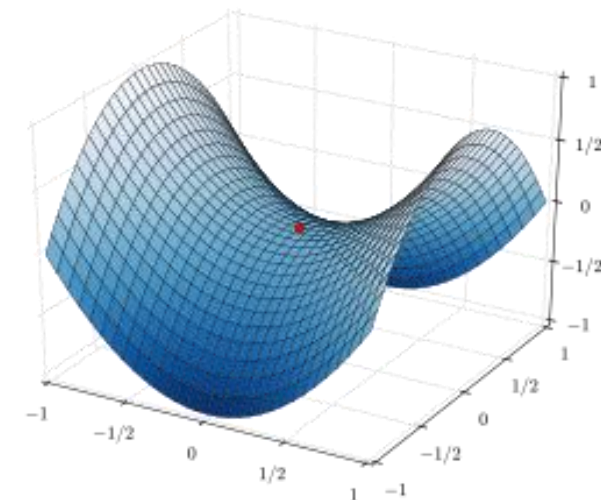
Gradient Descent

- Challenges with gradient descent
 - Local minima
 - Vanishing gradient
 - Exploding gradient
 - Saddle point
- Gradient descent is guaranteed to go **downhill**
 - Can not guaranteed a best or near best solution

The gradient of function $y = x^2$



Saddle point of
 $f(x, y) = x^2 - y^2$



Gradient Descent

- In spite of the local minima problem, deep learning produce rewarding results
 - Possibility of having many equally good solutions (local minima)
 - Possibility of very few local minima in the search space- specially on the high dimensional space

Gradient Descent

- Potential solution to local minima
 - If the performance is good/satisfactory → no need to worry
 - Retrain the model many times using different weights initialization and pick the best one
 - Use other methods - e.g., Stochastic Gradient Descent

Optimization

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent

Optimization

- Batch Gradient Descent: $\theta = \theta - \eta \cdot \Delta_{\theta} J(\theta)$
 - All samples in one iteration → iteration = epoch
 - Computes the gradients of the entire training dataset in each iteration and updates the parameters based on the average of these gradients
 - Advantages:
 - Higher chance of reaching global minima
 - Suitable for parallelization
 - stable updates
 - Disadvantages:
 - Computationally expensive
 - May get stuck in bad spot (local minima / saddle points)
 - Poor Generalization

Optimization

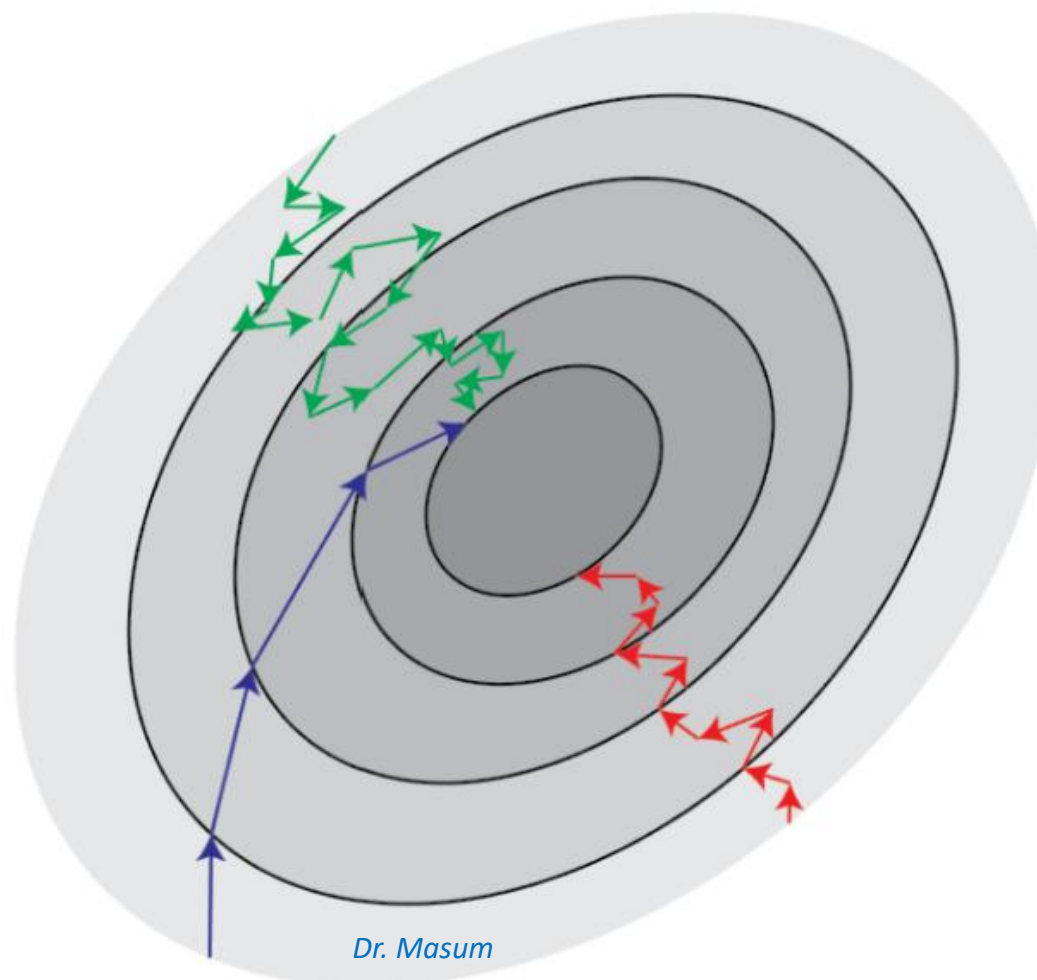
- Stochastic Gradient Descent: $\theta = \theta - \eta \cdot \Delta_{\theta} J(\theta; X^i, y^i)$
 - One random sample at each iteration
 - Performs weights update after each iteration for each sample
 - Number of iteration per epoch = number of samples
 - Advantages
 - Computationally efficient
 - Faster convergence
 - Can handle non-convex optimization problem
 - Disadvantages
 - Instability
 - May converge to a non-optimal solution
 - Higher chance of divergence -- Learning rate should be monitored carefully

Optimization

- Mini Batch Gradient Descent: $\theta = \theta - \eta \cdot \Delta_{\theta} J(\theta; X^{i:i+n}, y^{i:i+n})$
 - Take advantage of both batch and stochastic gradient descent
 - weights update after passing one batch in one iteration
 - Number of iteration per epoch = number of samples/batch size
 - Advantages
 - Faster convergence
 - Better generalization
 - Suitable for large datasets
 - Disadvantages
 - Adding additional hyperparameters (e.g., batch size)
 - Can be slower than SGD
 - May get stuck in bad spots

Optimization

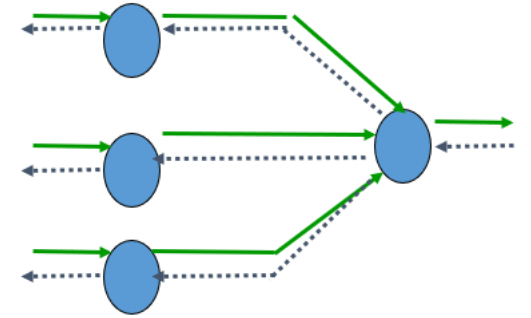
- Batch
- Mini-batch
- Stochastic



Dr. Masum

Backpropagation

- A network has millions of parameters
 - Backpropagation can be used to adjust weights efficiently
- Backpropagation
 - Apply to training set to search optimal set of parameters (weights) that minimize the total error of the samples of training set
 - Consists of two repeated parts: **Forward Pass & Backward Pass**
 - In the forward pass- error is calculated from input to output layer
 - In the backward pass- error is used for updating the weights layer by layer
 - Using calculating local gradient of each neuron



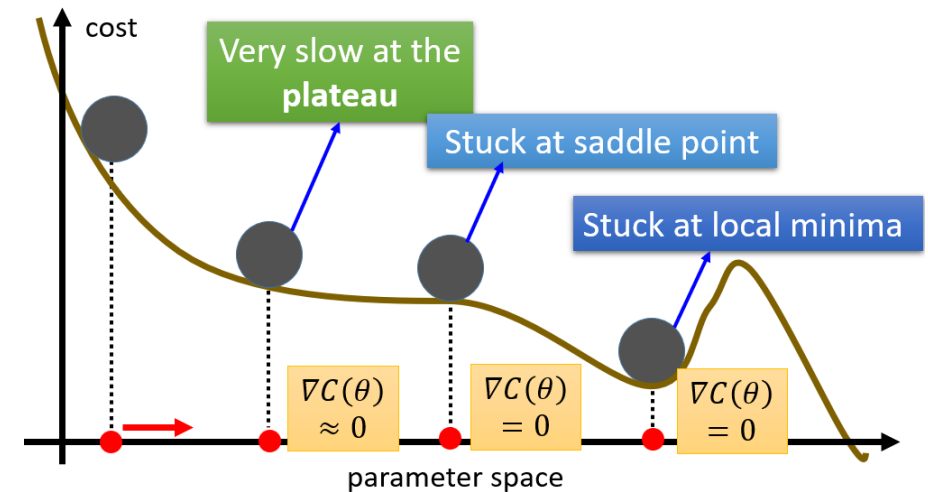
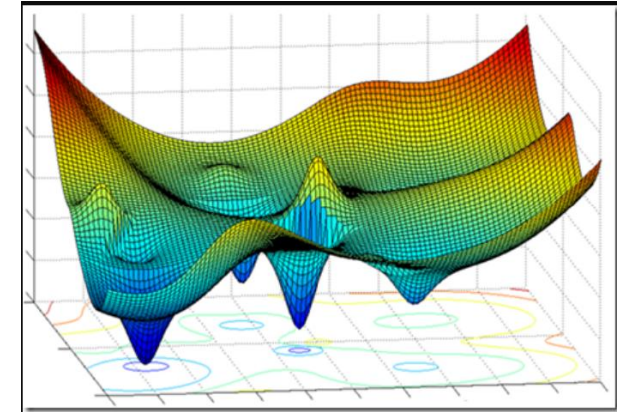
Backpropagation

- Chain Rule
- Partial derivative
- Derivative of Sigmoid function

Backpropagation

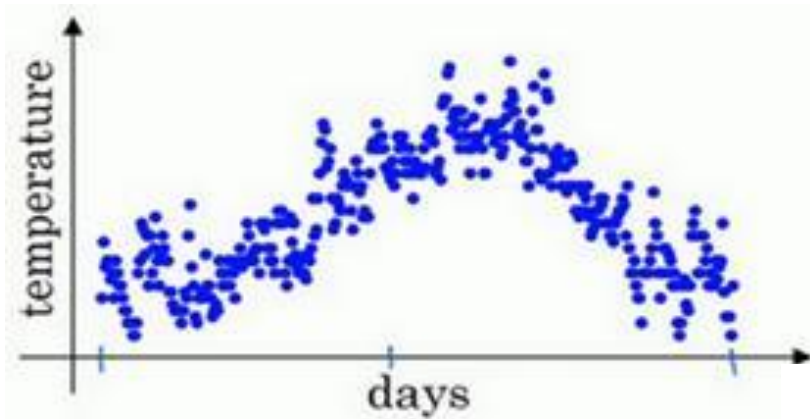
Advanced Optimization Methods

- Recall – Gradient Descent
 - Challenges with gradient descent
 - Local minima
 - Vanishing gradient
 - Exploding gradient
 - Saddle point
 - Gradient descent is guaranteed to go **downhill**
 - Can not guaranteed a best or near best solution

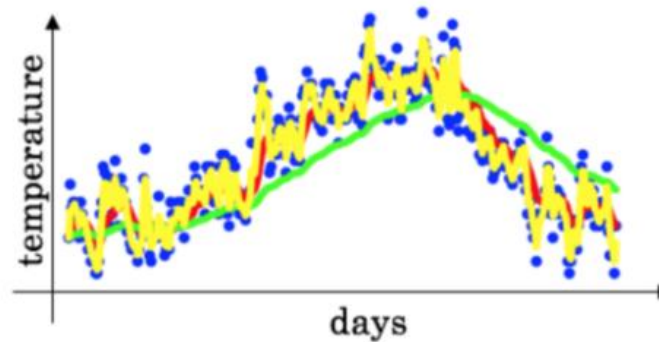


Advanced Optimization Methods

- Exponential Weighted Average

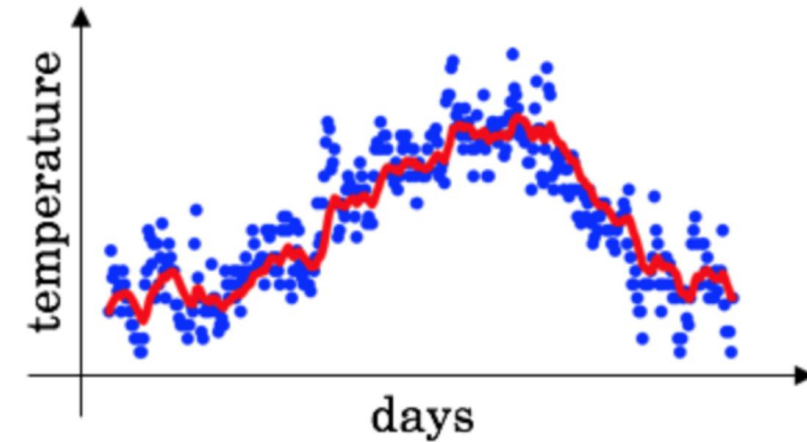


Time series: temperature vs. days



Averaging over 50 pts ($\beta = 0.98$) (green)

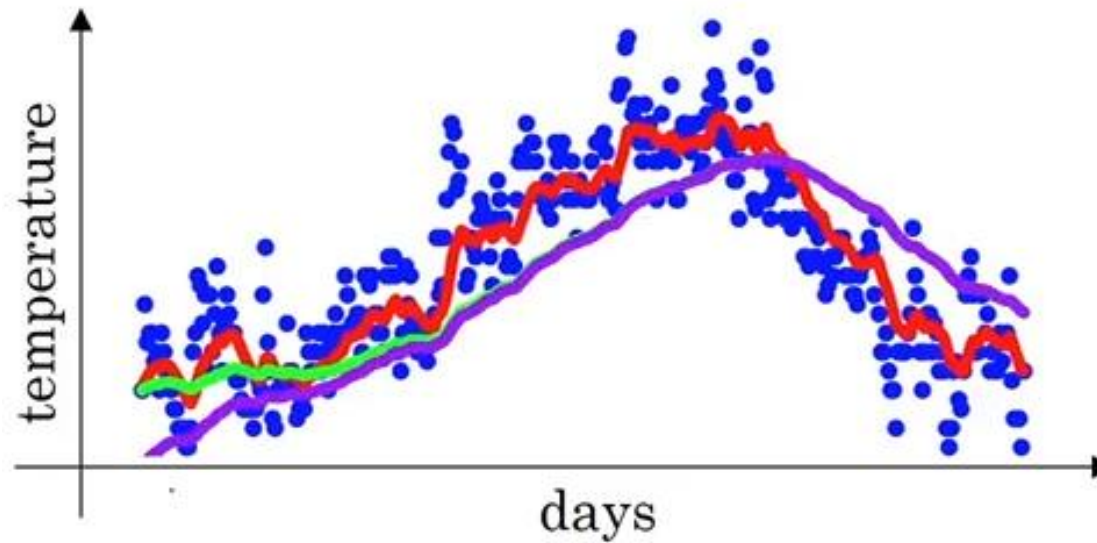
Averaging over 2 pts ($\beta = 0.5$) (yellow)



Averaging over 10 pts ($\beta = 0.9$)

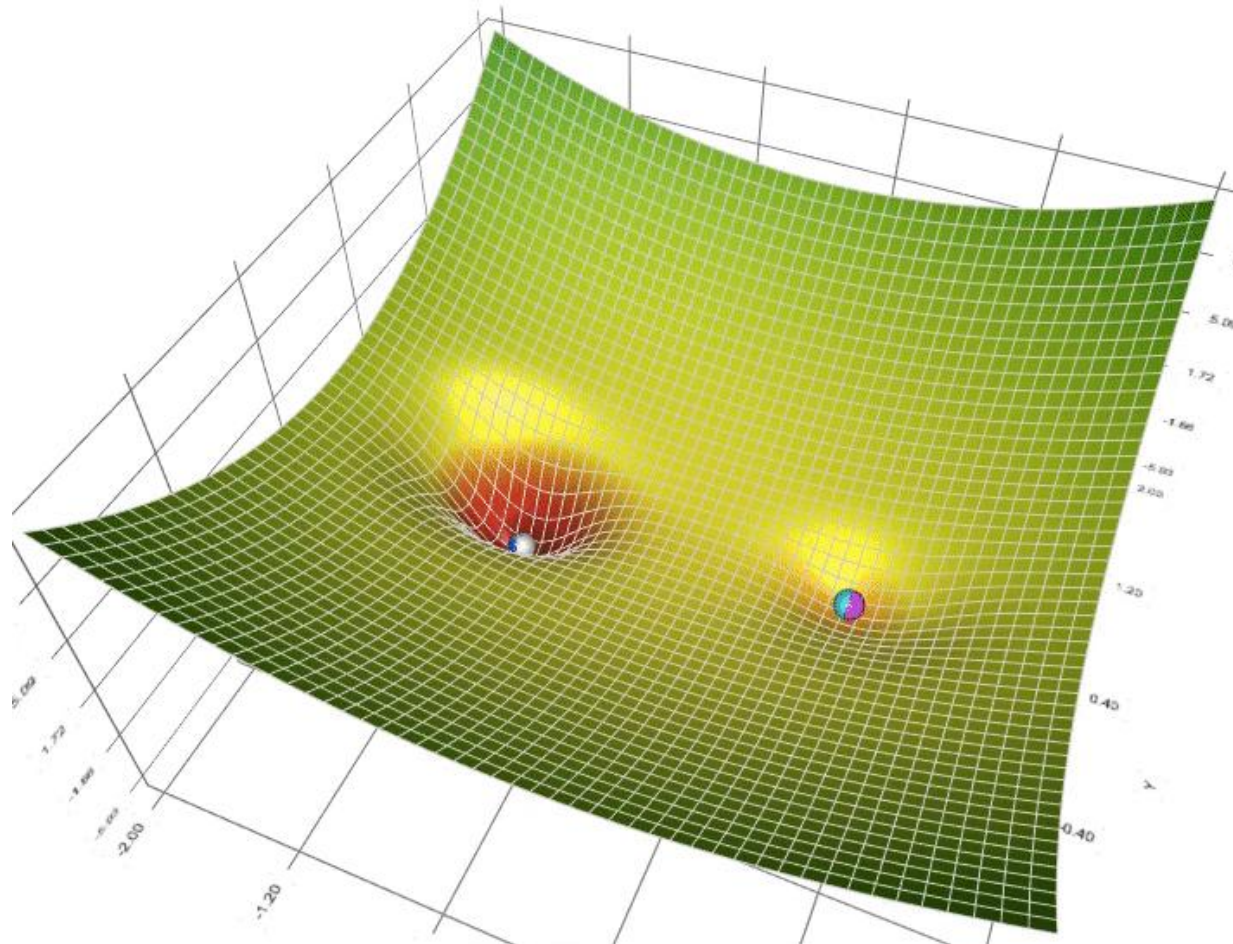
Advanced Optimization Methods

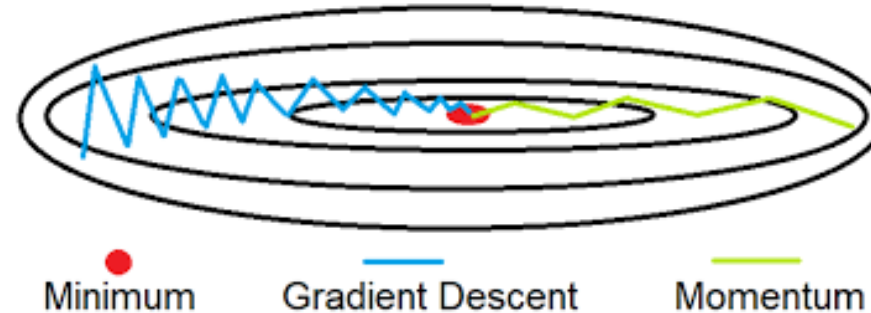
- Exponential Weighted Average- Bias correction



Advanced Optimization Methods

- GD (cyan),
- GD with momentum (magenta),
- AdaGrad (white),
- RMSProp (green)
- Adam (blue)





Gradient Descent with Momentum

- Gradient descent search can bounce back around the search space based on the gradient
- Momentum – is an extension to the gradient descent optimization algorithm
 - Accelerate the optimization process
 - Decrease the number of iterations to reach the optima → converges faster than gradient descent
 - Involves adding additional hyperparameter that controls the amount of history (momentum) to include the update equation
 - Momentum hyperparameter ranges from 0 to 1
 - Momentum 0 is the same as the gradient descent

```
keras.optimizers.SGD(
    learning_rate=0.01,
    momentum=0.0,
    nesterov=False,
    weight_decay=None,
    clipnorm=None,
    clipvalue=None,
    global_clipnorm=None,
    use_ema=False,
    ema_momentum=0.99,
    ema_overwrite_frequency=None,
    loss_scale_factor=None,
    gradient_accumulation_steps=None,
    name="SGD",
    **kwargs
)
```

- **momentum**: float hyperparameter ≥ 0 that accelerates gradient descent in the relevant direction and dampens oscillations. 0 is vanilla gradient descent. Defaults to 0.0.

Adam- Adaptive Moment Estimation

- Adam combines momentum accumulation with gradient values and incorporates the second moment (variance) of gradients.

- Computes a weighted average of past gradients, similar to gradient descent with momentum.

- Tracks the weighted average of past squared gradients to account for variance

- Corrects biases for both the first and second moments for improved optimization stability.

- The proposed default values are $\beta_1 = 0.9, \beta_2 = 0.999$, and $\epsilon = 10^{-8}$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

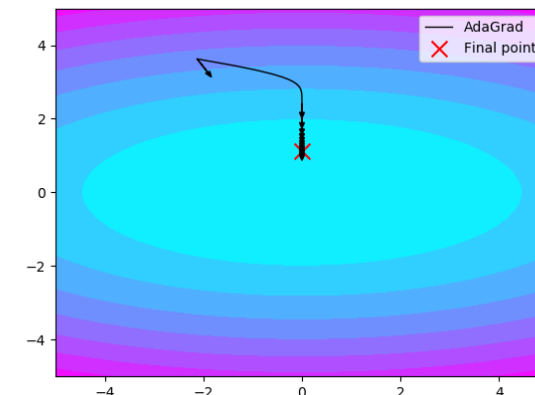
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

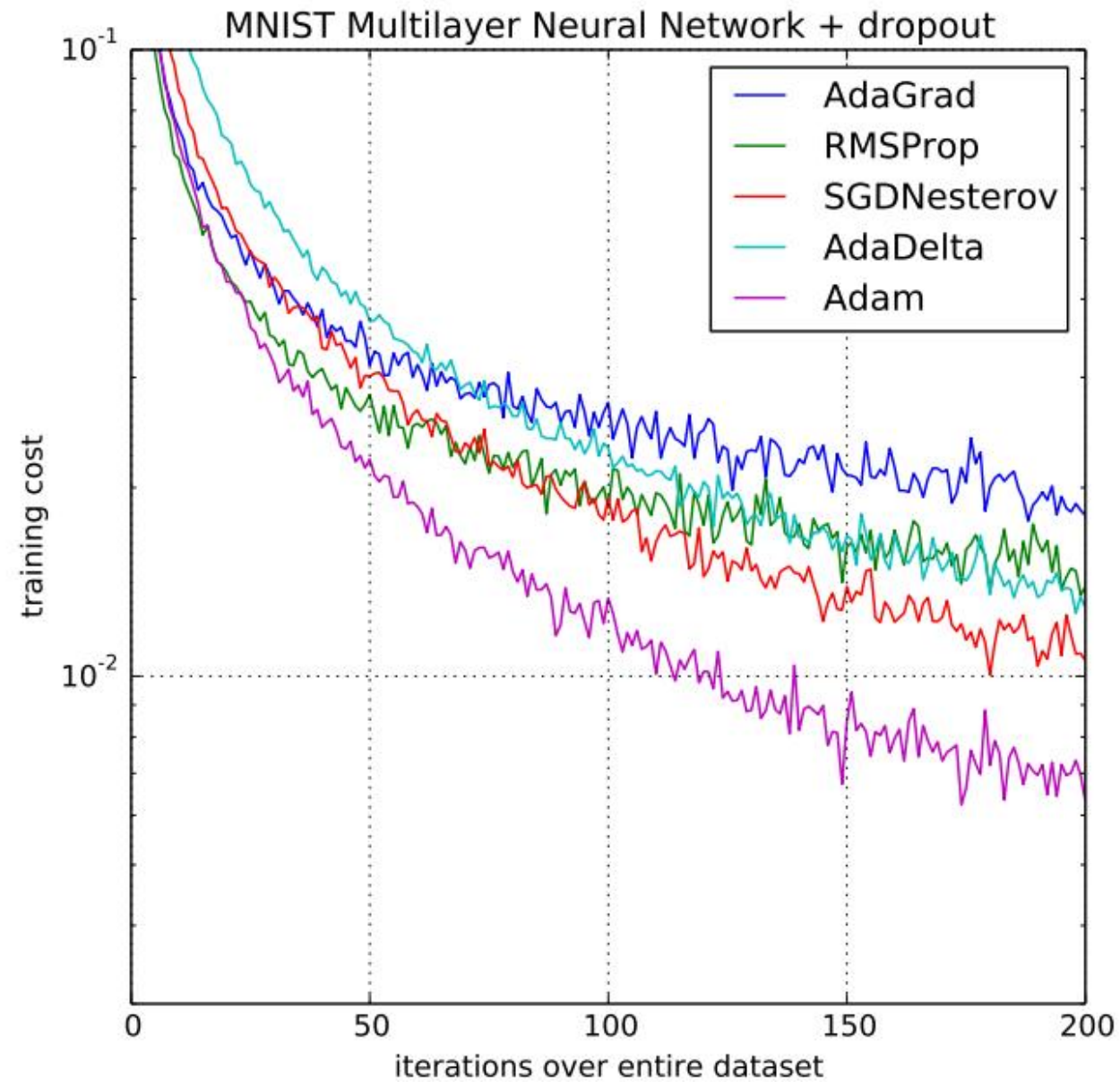
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

AdaGrad

- AdaGrad
 - adapts the learning rate
 - modifies the general learning rate at each time step for every parameter based on the past gradients
 - eliminates the need to manually tune the learning rate
 - however, the drawback is the accumulation of the squared gradients in the denominator
 - since every added term is positive, the accumulated sum keeps growing during training, causing the learning rate to shrink and becoming infinitesimally small.



AdaDelta



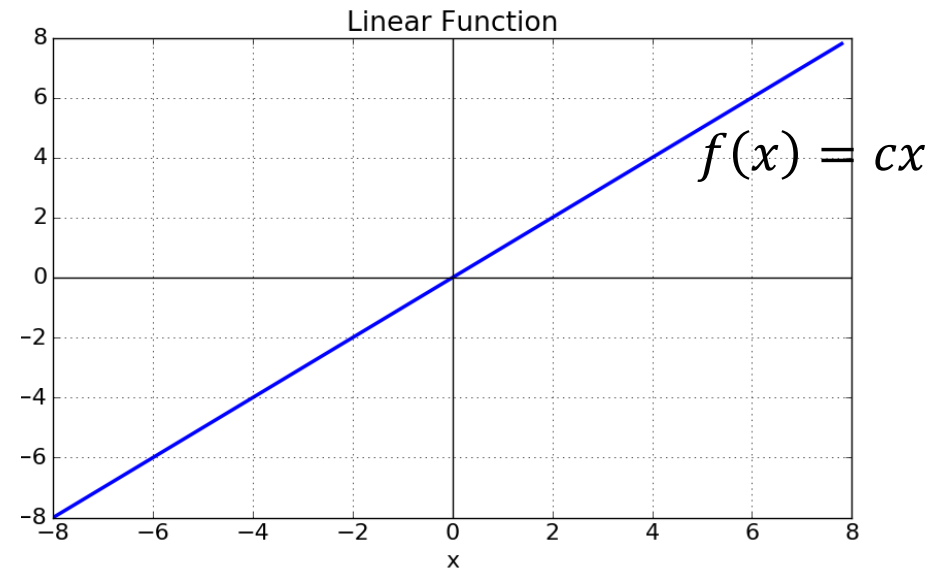
Activation Function

- The activation function's objective is to introduce nonlinearity into the network.
- Layers collapse without nonlinearities
 - The linear activation function (known as “**no activation**”) makes no changes to the weighted sum of the input; it just returns the value it was given
 - All deep learning models with linear activation functions, regardless of depth or width, are just single layer perceptron
- What makes an activation function good?
 - Nonlinearity
 - Computationally simple and efficient
 - Avoid limited range
 - Avoid vanishing/exploding gradient

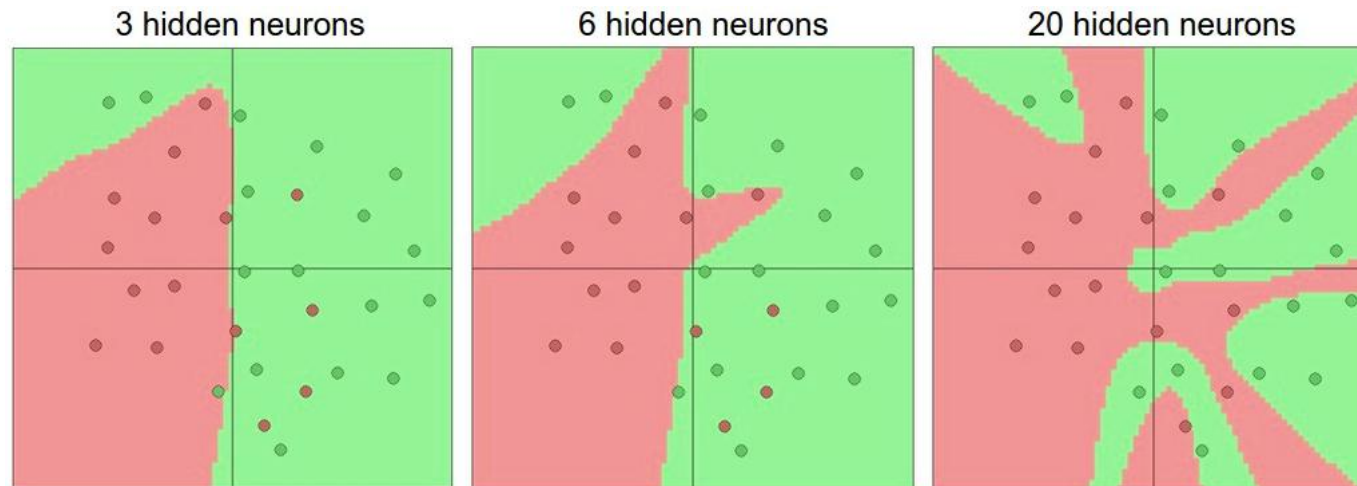
Activation Function

- What makes an activation function good?
 - Hidden layers:
 - A differentiable nonlinear function
 - Computationally simple and efficient
 - Avoid limited range
 - Avoid vanishing/exploding gradient
 - Output layer
 - Nonlinear (classification problem)
 - Linear (regression problem)

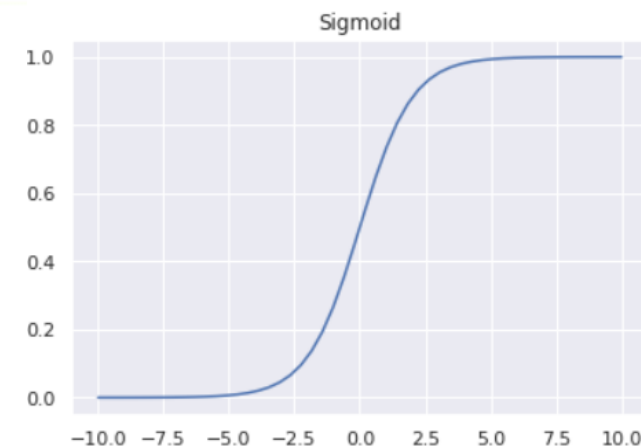
- Linear Function
 - Output is proportional to input signal
 - If value of $c = 1 \rightarrow f(x) = x \rightarrow$ identity activation function



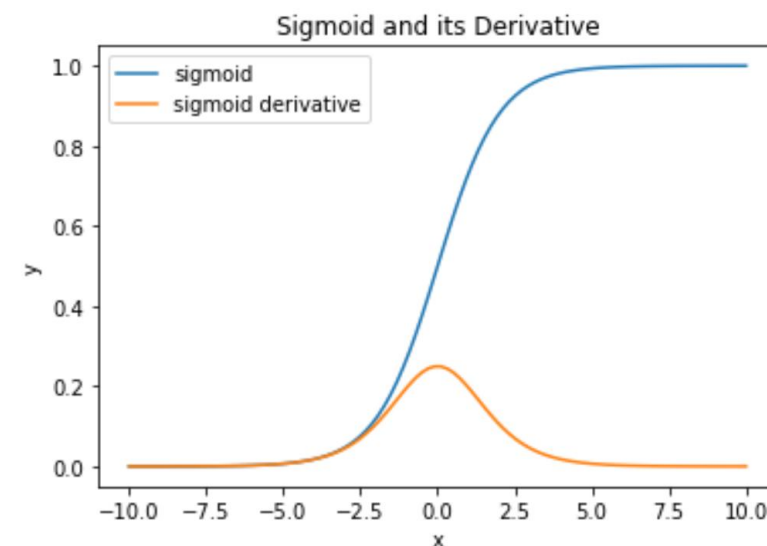
- Non-linear activation functions can learn complex (non-linear) pattern of data
 - With large number of layers and neurons can approximate more complex functions
 - May overfit



- Commonly used activation functions – Sigmoid
 - Takes any real valued numbers and pass a output of range [0,1]
 - Non-linear; however nearly linear around 0
 - Differentiable everywhere
 - Saturates at [0,1]
 - Great for output layer



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



- Softmax – converts raw logits (unnormalized scores) into a probability distribution over n classes.
- Ensures the outputs are non-negative and sum to 1, making it suitable for classification tasks.
 - Output Range: $[0, 1] \rightarrow$ probability of each class

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax

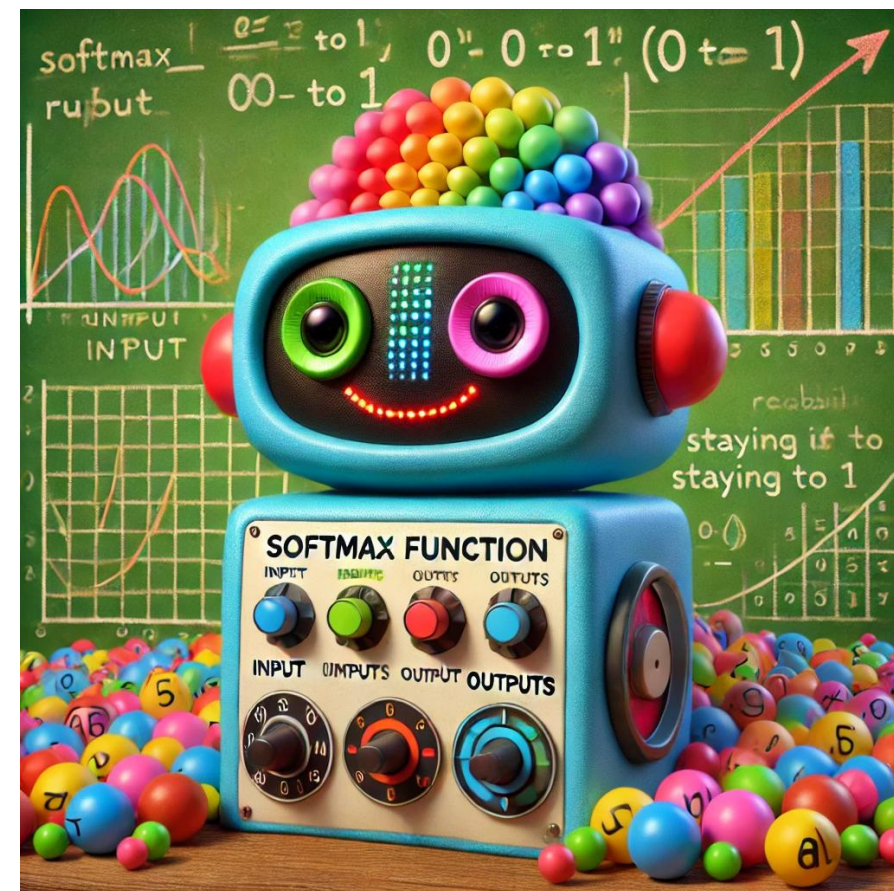
$\vec{z}$ = input vector

e^{z_i} = standard exponential function for input vector

K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

e^{z_j} = standard exponential function for output vector

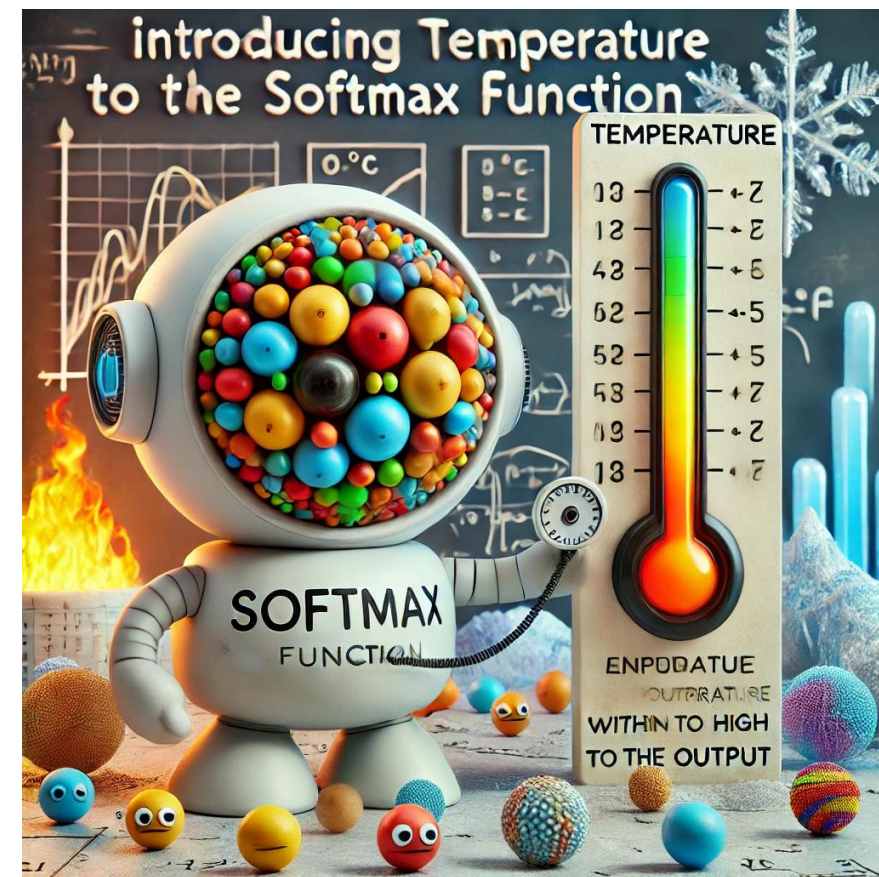


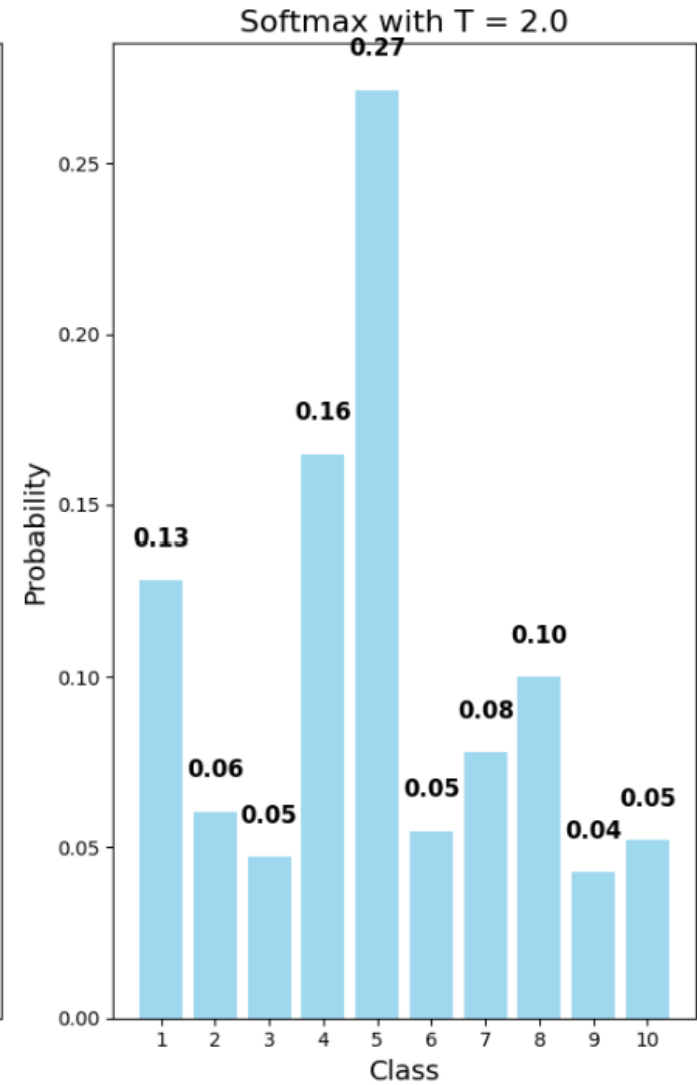
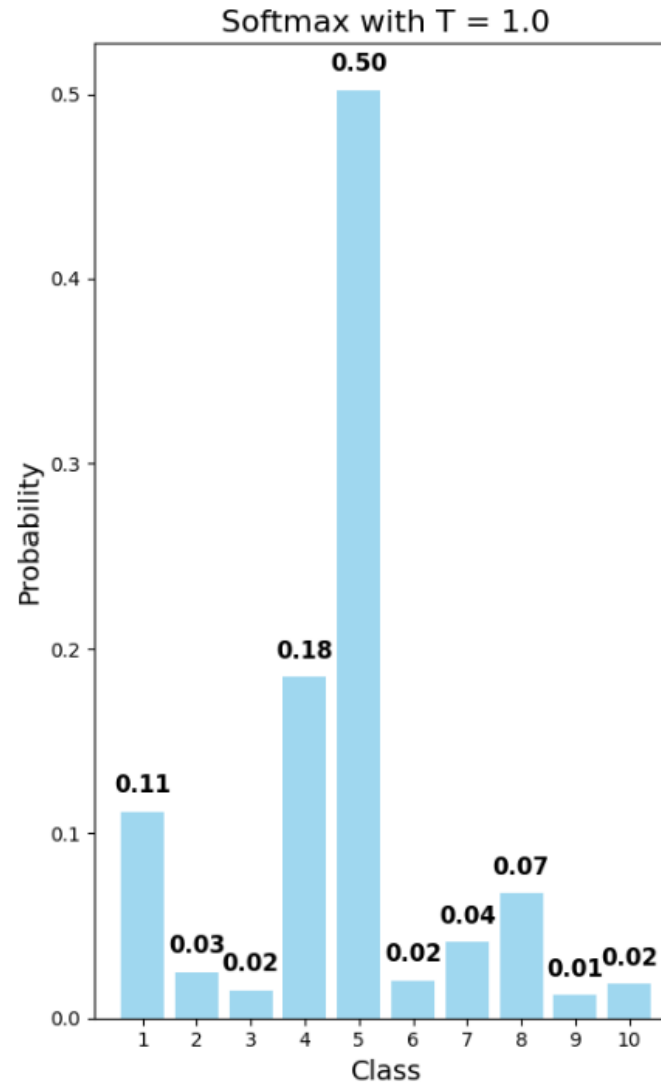
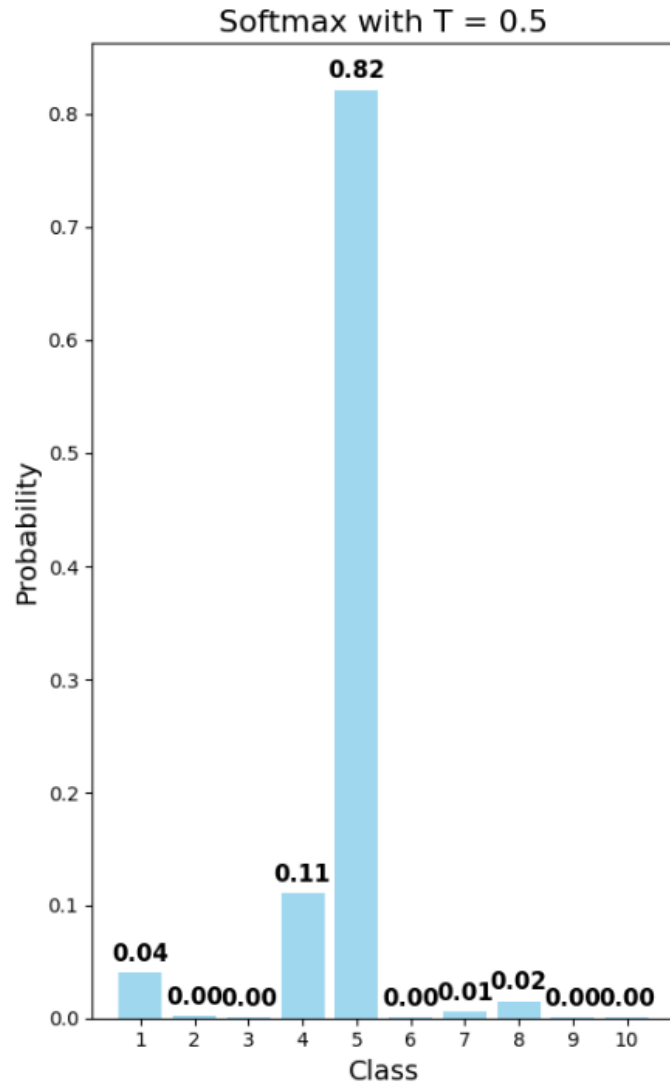
- Softmax with Temperature (T) – Temperature controls sharpness or smoothness of the output probabilities
 - Use cases: Softening probabilities during Knowledge distillation, analyzing uncertainty in model predictions

$$\text{Softmax with Temperature}(z_i, T) = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_{j=1}^n \exp\left(\frac{z_j}{T}\right)}$$

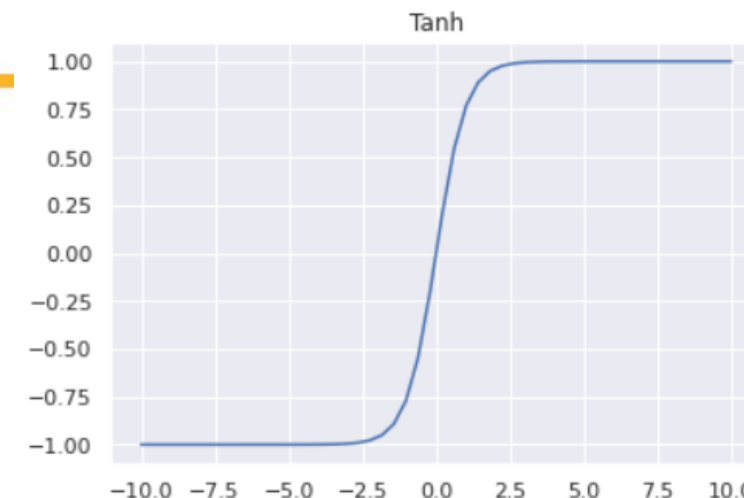
Discussion: Effect of T

- $T = 1$:
- $T > 1$:
- $T < 1$:

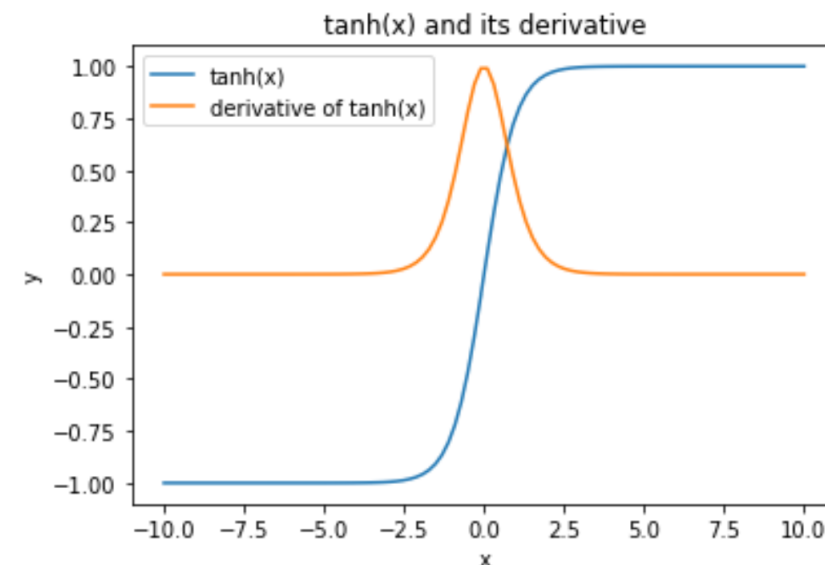




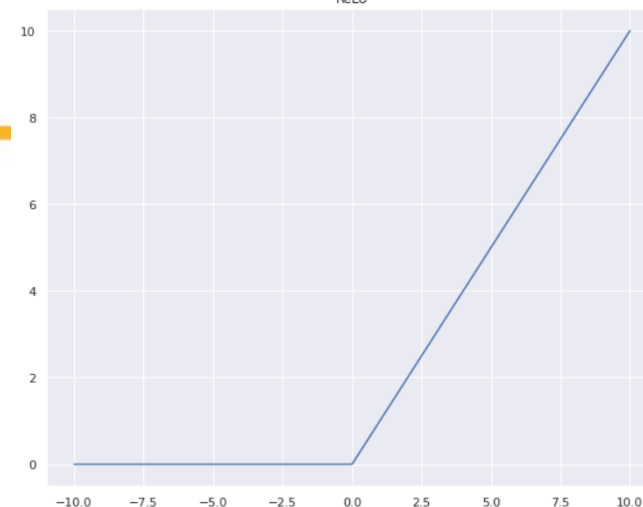
- Commonly used activation functions – Hyperbolic Tangent (Tanh)
 - Takes any real valued input and pass output of range $[-1,1]$
 - Non-linear; however nearly linear around 0
 - differentiable everywhere
 - Saturates at $[-1,1]$
 - Good for hidden layer



$$\sigma(x) = \frac{e^{-x} - e^x}{e^{-x} + e^x}$$

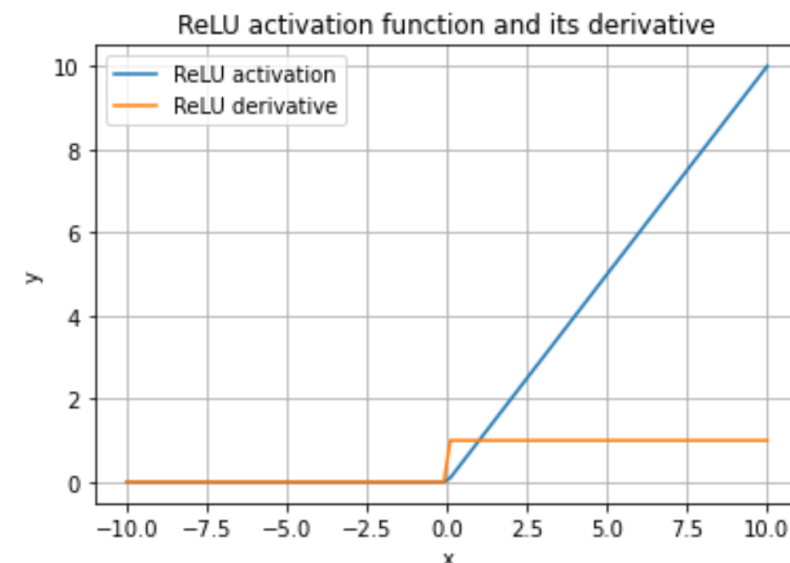


- Commonly used activation functions – Rectified Linear Unit (ReLU)
 - Takes a real valued number and threshold at zero
 - Non-linear
 - Deactivate all negative neurons → computationally efficient
 - Never saturates in the positive sides
 - Risk of exploding gradient
 - Solution – batch normalization
 - Not differentiable everywhere
 - Derivative is undefined at the corner point
 - Solution ?
 - great for hidden layer

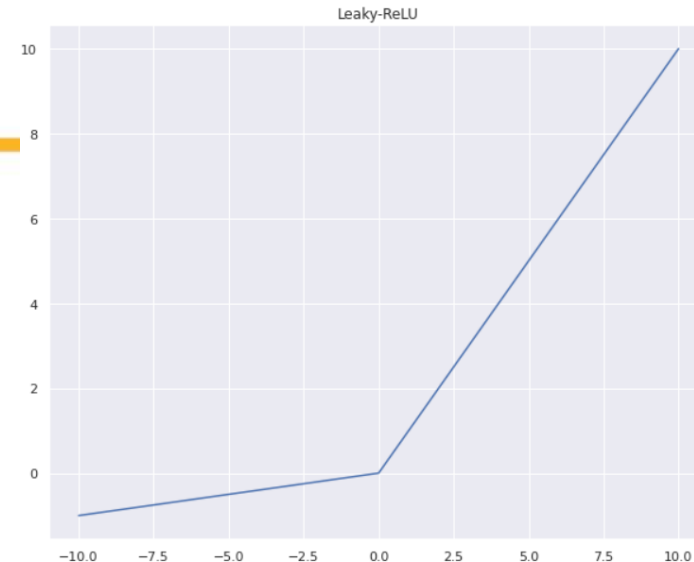


$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

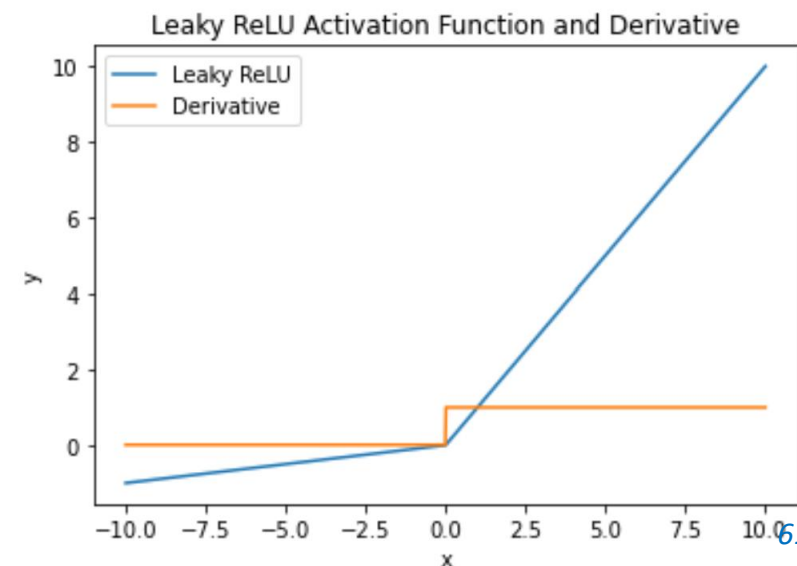


- Variants of ReLU activation functions – Leaky ReLU
 - Resolves the dying ReLU problem
 - Non-linear;
 - Never saturates
 - Risk of exploding gradient
 - Solution – batch normalization
 - Negative neurons can be activated
 - Typical $\alpha = 0.1$

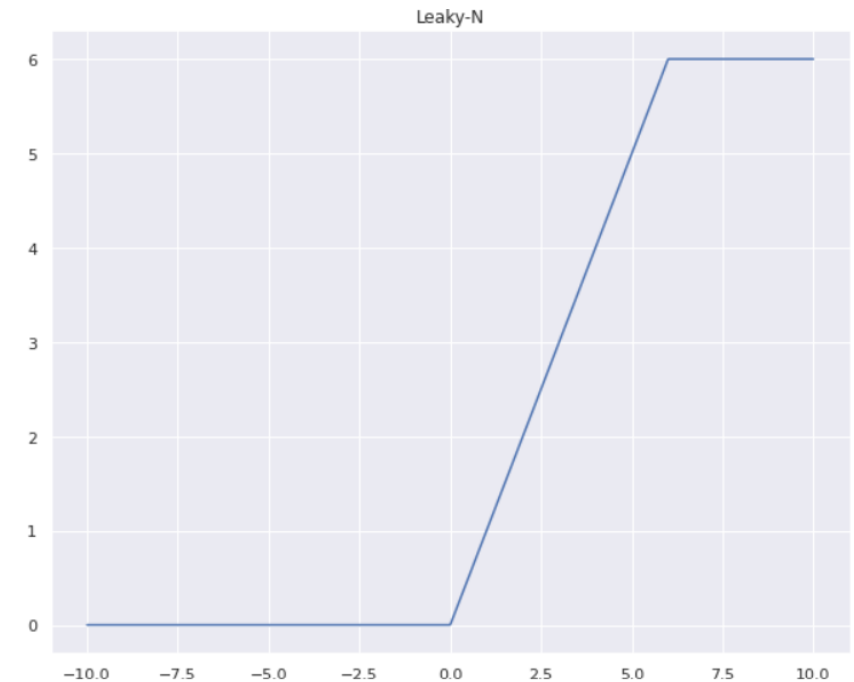


$$f(x) = \max(\alpha x, x)$$

$$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$



- Variants of ReLU activation functions – ReLU-N
 - Non-linear;
 - Saturates both sides
 - No Risk of exploding gradient
 - Typical N = 6



$$\sigma(x) = \min(N, \max(0, x))$$

- How to choose activation function for hidden layers?
 - Same activation function in all hidden layers
 - Based on the type of neural network architecture
 - MLP, CNN → ReLU
 - Recurrent → Tanh/sigmoid
- How to choose activation function for output layers?
 - Based on the prediction problem
 - Linear → Regression
 - Sigmoid → binary classification
 - Softmax → multi-class classification

- **GELU (Gaussian Error Linear Unit)** – provides a **smooth, differentiable approximation** of ReLU with improved gradient behavior.

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}(x/\sqrt{2}) \right]$$

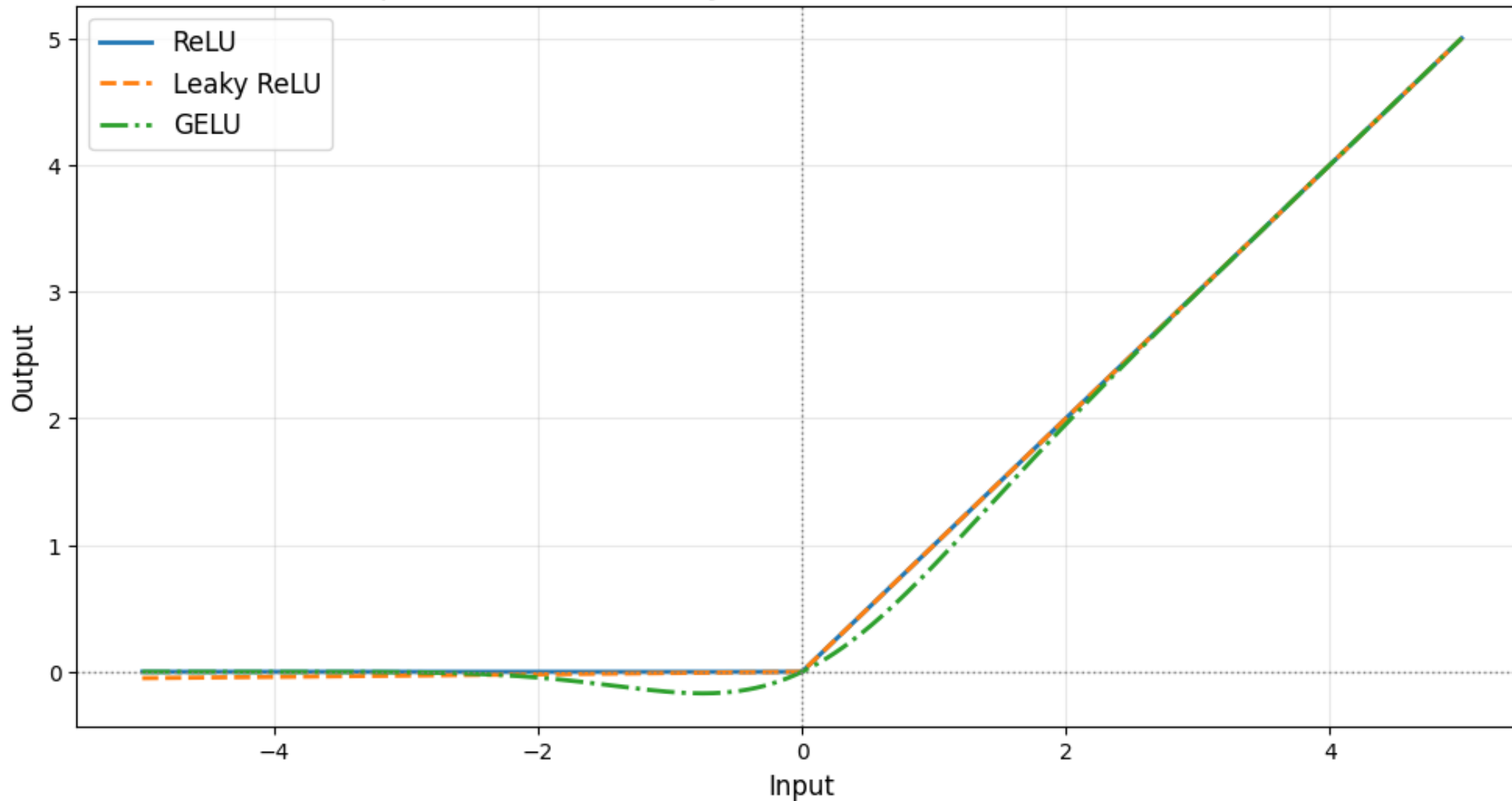
We can approximate the GELU with

$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

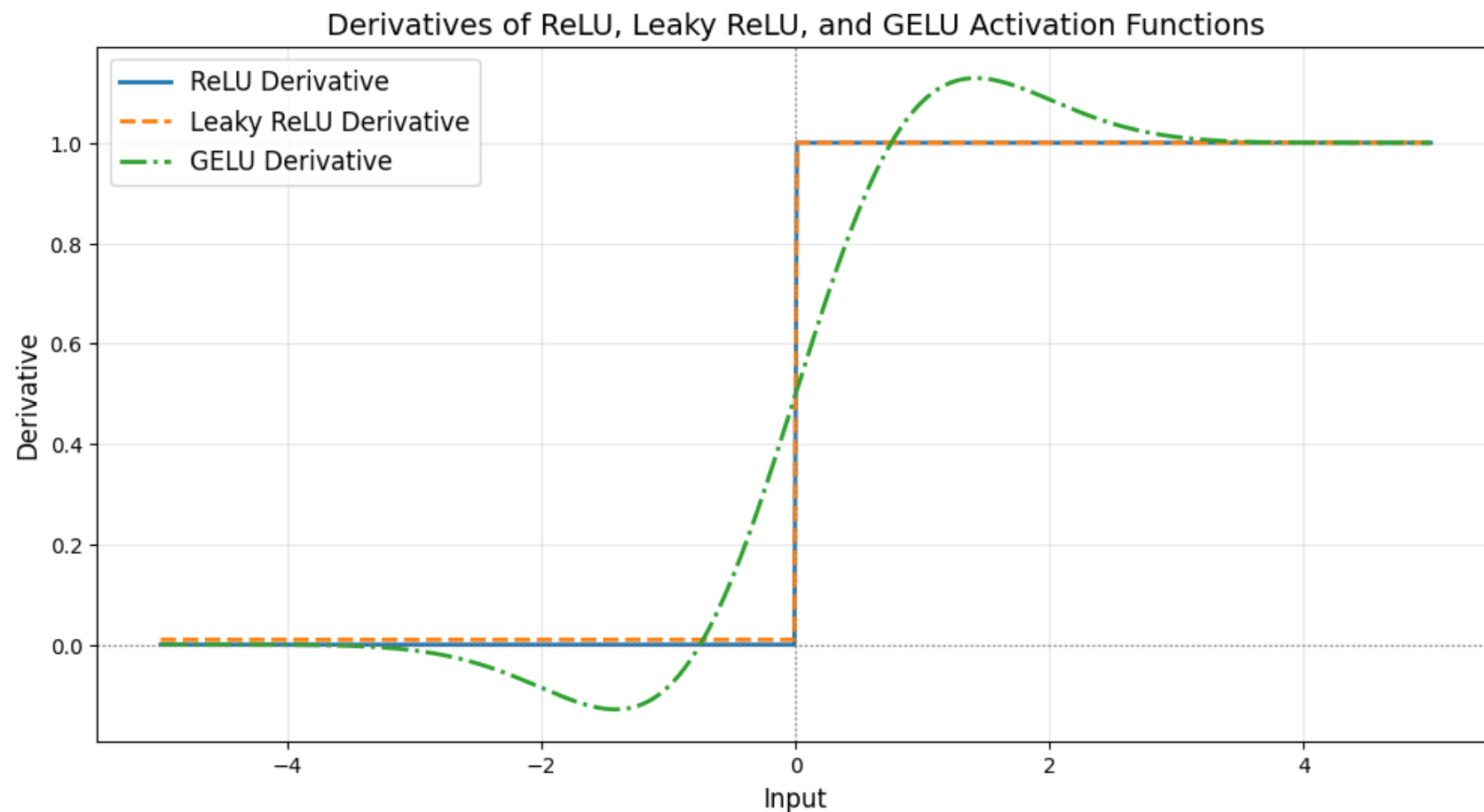
- **No Dead Neurons:** GELU does not abruptly set negative inputs to zero;
 - For positive inputs, GELU behaves similarly to ReLU but with a smoother transition.
 - For negative inputs, it applies a diminishing effect instead of outright discarding them.
- Used extensively in **transformers** (e.g., GPT, BERT, T5)



Comparison of ReLU, Leaky ReLU, and GELU Activation Functions



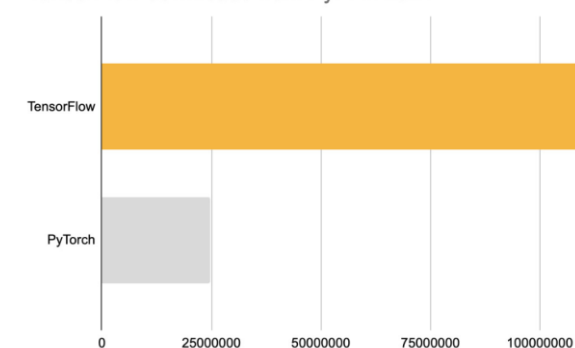
- **ReLU Derivative:** Constant 1 for positive inputs, 0 for negative inputs and discontinuous at $x = 0$
- **Leaky ReLU Derivative:** Constant 1 for positive inputs, for negative inputs, the gradient is α (e.g., 0.01), allowing a small gradient flow and avoiding dead neurons.
- **GELU Derivative:** Smooth and continuous across all inputs.



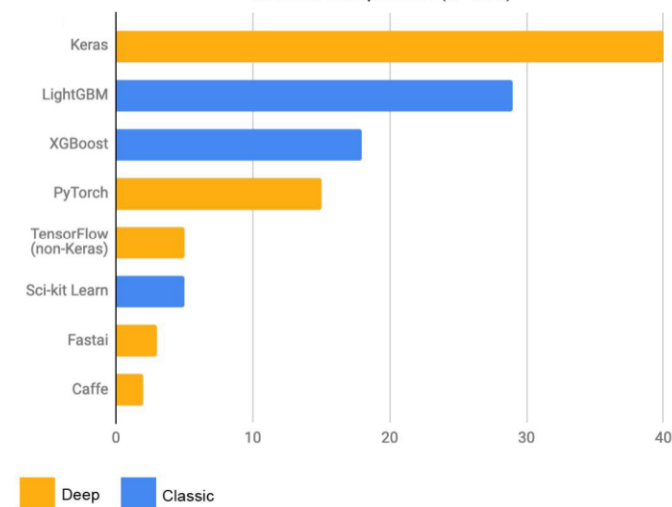
DL Frameworks

- Keras
 - Built on top of TensorFlow 2
 - Can scale to large clusters of GPUs
 - A vast ecosystem
 - Covers every step of ML: data management to hyperparameter tuning to model deployment
 - Easy to design DL
 - Can give more focus on research
 - Many example code given for DL

TensorFlow downloads from PyPI in 2021



Primary ML software tool used by **top-5 teams** on Kaggle in each competition (n=120)



DL Frameworks

- Keras – Sequential model
 - Appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor
 - Specify the input shape
 - Model summary

```
model = keras.Sequential()
model.add(layers.Dense(2, activation="relu"))
model.add(layers.Dense(3, activation="relu"))
model.add(layers.Dense(4))
```

```
model = keras.Sequential()
model.add(layers.Dense(2, activation="relu", input_shape=(4,)))

model.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 2)	10
Total params: 10		
Trainable params: 10		
Non-trainable params: 0		

DL Frameworks

- Keras – Sequential model
 - Train model

Train the model

```
batch_size = 128
epochs = 15

model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, validation_split=0.1)
```

- Evaluate the trained model

Evaluate the trained model

```
score = model.evaluate(x_test, y_test, verbose=0)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```

```
Test loss: 0.023950600996613503
Test accuracy: 0.9922000169754028
```

Design Your Own ANN

- Demo – MNIST digit recognition



Design Your Own ANN

- Demo – MNIST digit recognition

Type	Classifier	Distortion	Preprocessing	Error rate (%)
Linear classifier	Pairwise linear classifier	None	Deskewing	7.6 ^[10]
K-Nearest Neighbors	K-NN with non-linear deformation (P2DHMDM)	None	Shiftable edges	0.52 ^[26]
Boosted Stumps	Product of stumps on Haar features	None	Haar features	0.87 ^[27]
Non-linear classifier	40 PCA + quadratic classifier	None	None	3.3 ^[10]
Random Forest	Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC) ^[28]	None	Simple statistical pixel importance	2.8 ^[29]
Support-vector machine (SVM)	Virtual SVM, deg-9 poly, 2-pixel jittered	None	Deskewing	0.56 ^[30]
Deep neural network (DNN)	2-layer 784-800-10	None	None	1.6 ^[31]
Deep neural network	2-layer 784-800-10	Elastic distortions	None	0.7 ^[31]
Deep neural network	6-layer 784-2500-2000-1500-1000-500-10	Elastic distortions	None	0.35 ^[32]
Convolutional neural network (CNN)	6-layer 784-40-80-500-1000-2000-10	None	Expansion of the training data	0.31 ^[33]
Convolutional neural network	6-layer 784-50-100-500-1000-10-10	None	Expansion of the training data	0.27 ^[34]
Convolutional neural network (CNN)	13-layer 64-128(5x)-256(3x)-512-2048-256-256-10	None	None	0.25 ^[20]
Convolutional neural network	Committee of 35 CNNs, 1-20-P-40-P-150-10	Elastic distortions	Width normalizations	0.23 ^[15]
Convolutional neural network	Committee of 5 CNNs, 6-layer 784-50-100-500-1000-10-10	None	Expansion of the training data	0.21 ^{[22][23]}
Random Multimodel Deep Learning (RMDL)	10 NN-10 RNN - 10 CNN	None	None	0.18 ^[25]
Convolutional neural network	Committee of 20 CNNs with Squeeze-and-Excitation Networks ^[35]	None	Data augmentation	0.17 ^[36]

Design Your Own ANN

- Pytorch

```
## To match the shape of y :
model = torch.nn.Sequential(
    torch.nn.Linear(3, 1),
    torch.nn.Flatten(0, 1)
)

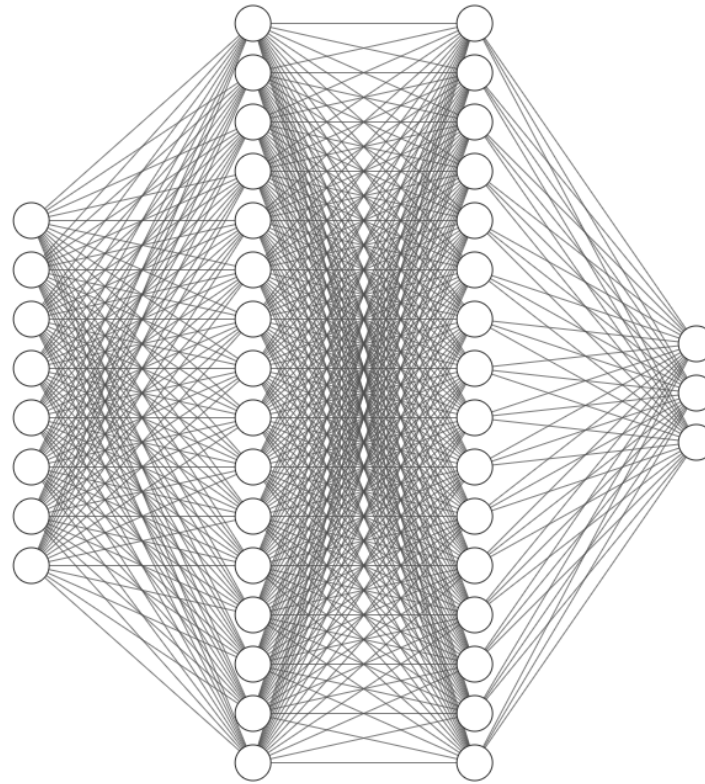
# The nn package also contains definitions of popular loss functions; in this
# case we will use Mean Squared Error (MSE) as our loss function.
loss_fn = torch.nn.MSELoss(reduction='sum')

learning_rate = 1e-6
for t in range(2000):

    # Forward pass: compute predicted y by passing x to the model. Module objects
# override the __call__ operator so you can call them like functions. When
# doing so you pass a Tensor of input data to the Module and it produces
# a Tensor of output data.
    y_pred = model(xx)
```

Model depth vs. breadth

- What is the width and depth of the model?



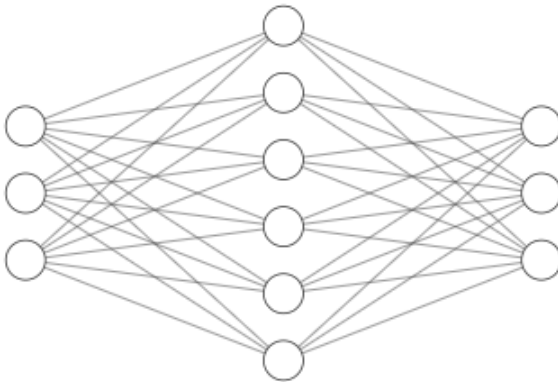
Model depth vs. breadth

- Count the total number of trainable parameters
- Can same number of nodes have different number of parameters?
- Are deeper models necessarily better?

Wider model

nodes 9

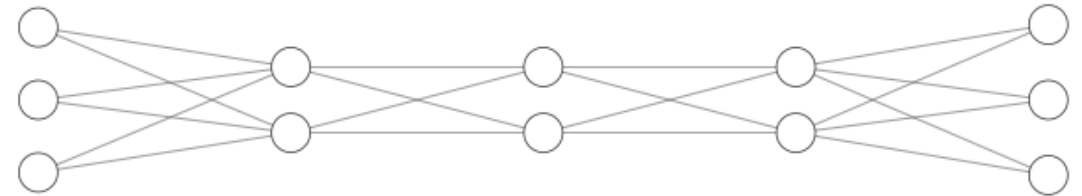
trainable parameters?



deeper model

nodes 9

trainable parameters?

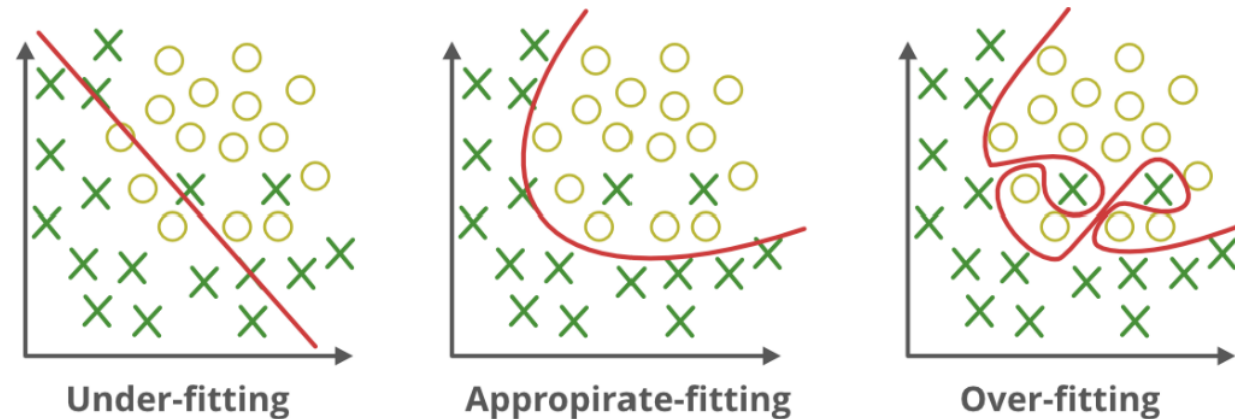


Model depth vs. breath

- A shallow networks with very wide layers may be very good at memorization but high chance of not generalizing well
- A deeper network – take the advantage of learning features in successive layers
 - Think –CNN
- What if we build a very deep and very wide network?

Overfitting

- Overfitting
 - Situation where model works well with training data, but fails for test data
 - High accuracy for training data, low accuracy for test data



Overfitting



DeepLearning.AI
@DeepLearningAI_

...

"I understood that reference!" - Steven G. Rogers

Created by: Machine Learning Memes for
Convolutional Teens

My model on training data



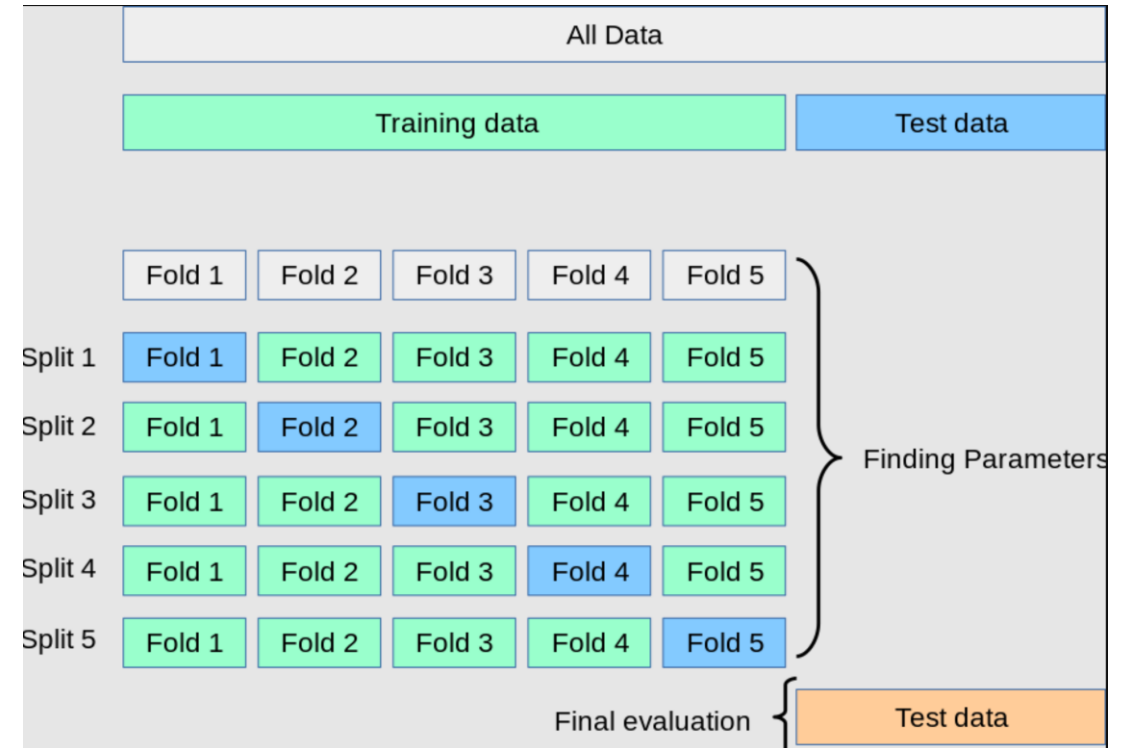
My model on test dataset



8:00 AM · Aug 3, 2020 · HubSpot

Overfitting

- Cross-validation
 - What happen if K is too large?
 - What happen if K is too small?
 - What is stratified k-fold CV?



Regularization

- Regularization
 - A solution to overfitting problem
 - Penalizes “memorization” (over-learning examples)
 - Helps the model to generalize to unseen (test) data
 - Can change the representation of the model
 - E.g., more sparse network
 - Can increase or decrease training time based on the regularizer
 - Tradeoff between training vs. test accuracy
 - Increase generalization
 - Generally, works better for larger network and with sufficient data

Regularization

- Commonly used regularizer:
 - Node regularizer - Dropout – modify model architecture
 - Loss regularizer - L1/L2 – modify loss function
 - Data regularizer – data augmentation

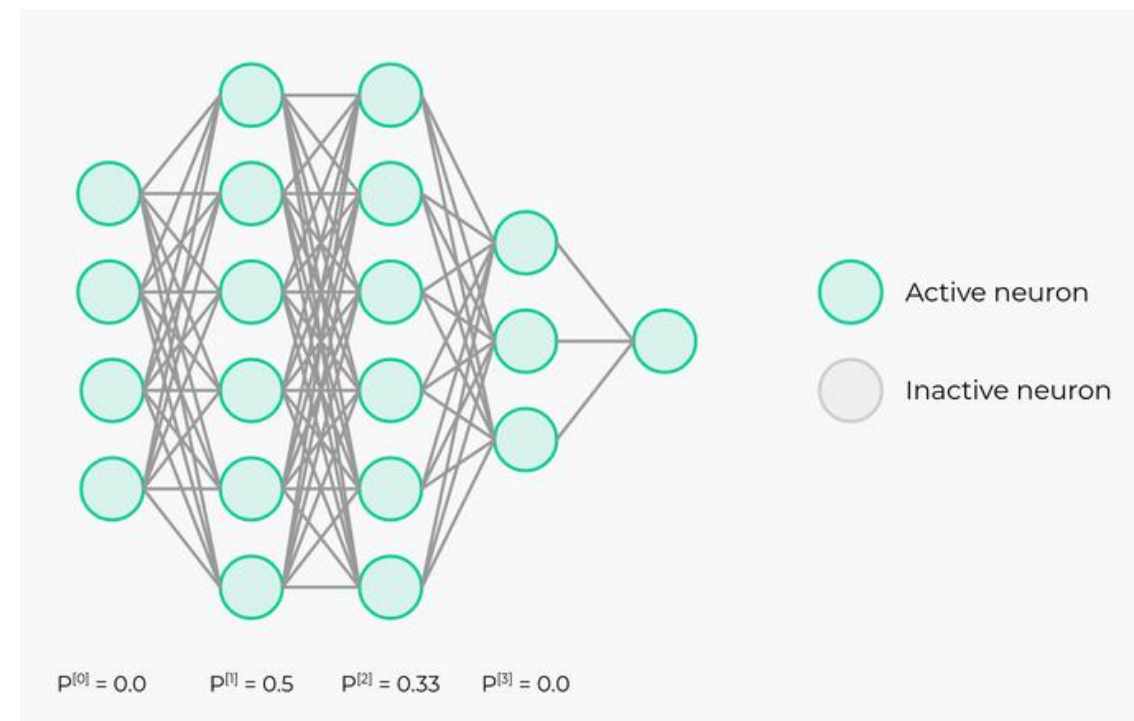
Regularization

- **Dropout** - a simple way to prevent neural network from overfitting¹
 - Deep neural networks with many parameters are extremely powerful machine learning systems
 - However, overfitting is a serious problem in such networks
 - Large networks are also slow to use, making it difficult to deal with overfitting by combining the predictions of many different large neural nets at test time
 - Dropout is a technique for addressing this problem

1. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1), 1929-1958.

Regularization

- Key idea of Dropout
 - randomly drop units (along with their connections) from the neural network during **training**
 - This prevents units from **co-adapting** too much
 - A specific neuron may learn to depend on one (or more) of the previous neuron; therefore, removing the neuron prevents the dependency and forced to learn information from other neurons (features)



Regularization

- Key idea of Dropout
 - During training, dropout samples from an exponential number of different “thinned” networks
 - At test time, it is easy to approximate the effect of averaging the predictions of all these **thinned** networks by simply using a **single unthinned** network that has smaller weights
 - This significantly reduces overfitting and gives major improvements over other regularization methods

- Key idea of Dropout

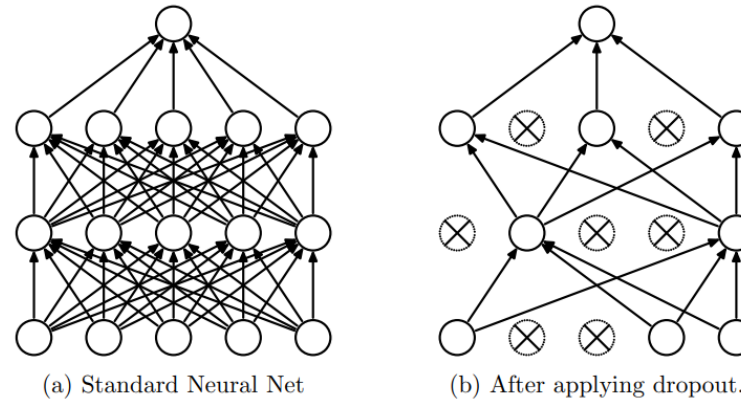


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

```
keras.layers.Dropout(rate, noise_shape=None, seed=None, **kwargs)
```

Applies dropout to the input.

The **Dropout** layer randomly sets input units to 0 with a frequency of **rate** at each step during training time, which helps prevent overfitting. Inputs not set to 0 are scaled up by $1 / (1 - \text{rate})$ such that the sum over all inputs is unchanged.

Note that the **Dropout** layer only applies when **training** is set to **True** in **call()**, such that no values are dropped during inference. When using **model.fit**, **training** will be appropriately set to **True** automatically. In other contexts, you can set the argument explicitly to **True** when calling the layer.

Regularization

- Key idea of Dropout
 - Gradients are computed only backpropagation during training

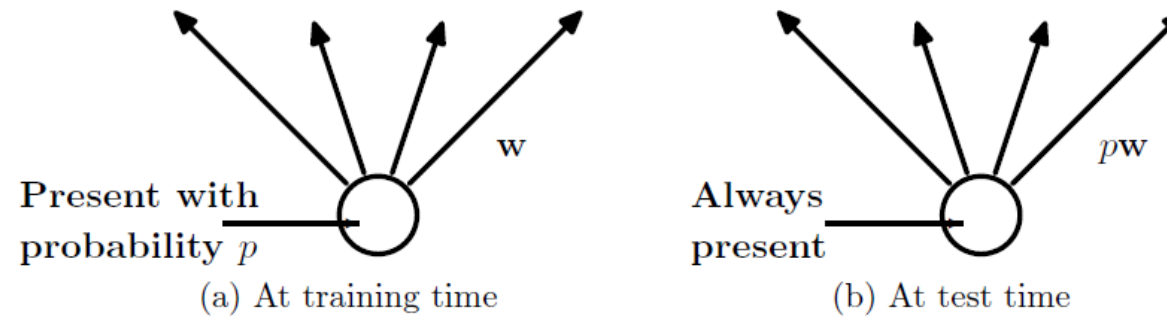
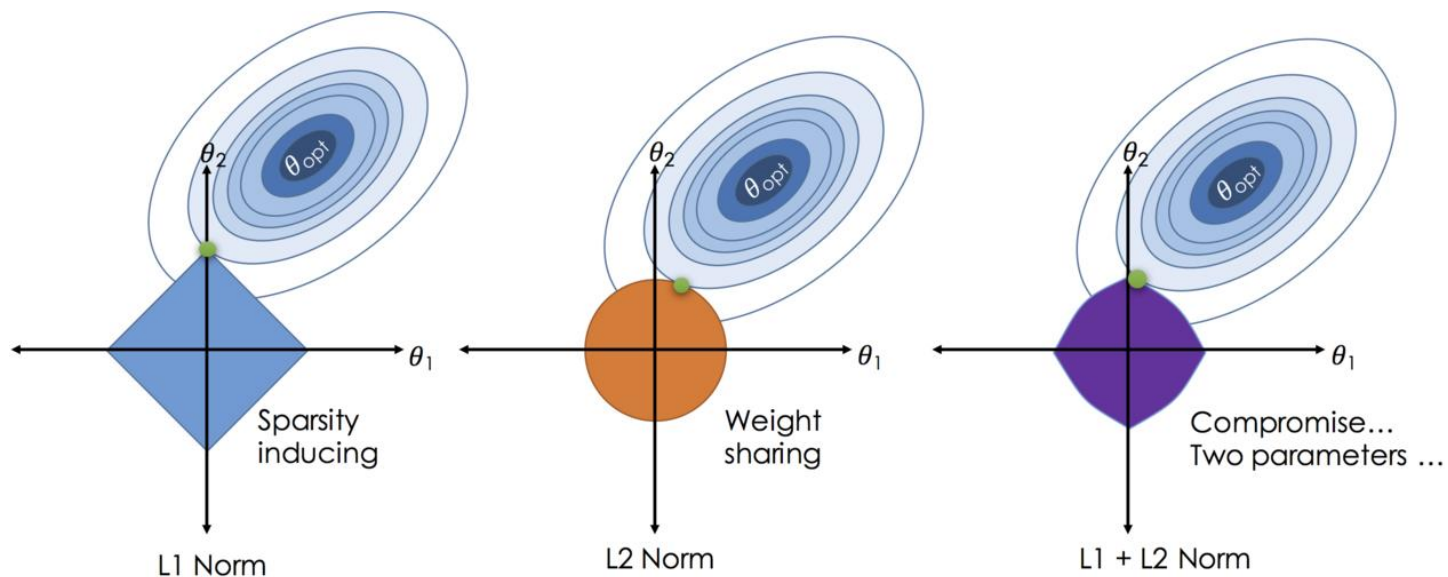


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights w . **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

Regularization

- Why dropout works
 - Intuition – Can't rely on any one feature, so have to spread out weights¹
- How to find the optimal dropout rate?
 - Hyperparameter tuning
 - However, 0.5 generally works well
- Dropping inputs is not a common practice, though you can experiment

- Weight regularization (L1/L2)
 - A known feature selection method (in ML term embedded feature selection) which causes non-relevant features to have smaller weights (in some cases zero weights), to penalize their contribution towards training
 - Commonly used in ML
 - L1 regularization → Lasso
 - L2 regularization → ridge regression



Regularization

- In DL, main goal is to minimize the cost by achieving optimal set of learnable parameters
- **L1** regularization – adds **absolute** value of magnitude of all learnable parameters as penalty term to the loss function
- **L2** regularization – adds **square** magnitude of all learnable parameters as penalty term to the loss function

$$W = \underset{W}{\operatorname{argmin}} J$$

$$J = \frac{1}{n} \sum_{i=1}^n L(\widehat{y}_i, y_i)$$

$$J = \frac{1}{n} \sum_{i=1}^n L(\widehat{y}_i, y_i) + \lambda ||W||_2^2$$

$$J = \frac{1}{n} \sum_{i=1}^n L(\widehat{y}_i, y_i) + \lambda ||W||_1$$

Regularize as little as possible

Regularization - Batch Normalization

- Normalization - scaling all features in the same range
 - higher values dominates lower values during training
 - Lower values features become useless
 - Exploding gradient may occur with the higher valued features
- Popular normalization techniques
 - Minmax normalization $\rightarrow x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$
 - Standardization $\rightarrow x_{scaled} = \frac{x - mean}{std}$

Regularization - Batch Normalization

- Batch Normalization (BN)
 - A technique to normalize activations in intermediate layers
 - Improve accuracy and speed up training

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
 Parameters to be learned: γ, β
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Regularization - Batch Normalization

- Why Batch Normalization (BN) works
 - The exact reasons for BN's effectiveness are still not cleared
 - Reducing internal covariate shift – initially claimed [1]
 - Network training converges faster if its inputs are whitened - linearly transformed to have zero means and unit variances
 - BN (whitening all the consecutive layers) achieves fixed distributions of inputs that removing the ill effects of the internal covariate shift
 - However, later, researchers claimed that BN makes more fundamental impact on the training process [2]
 - It makes the optimization landscape significantly smoother
 - This smoothness, brings a more predictive and stable behavior of the gradients, allowing for faster training

[1] Ioffe, S., & Szegedy, C. (2015, June). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448-456). PMLR.

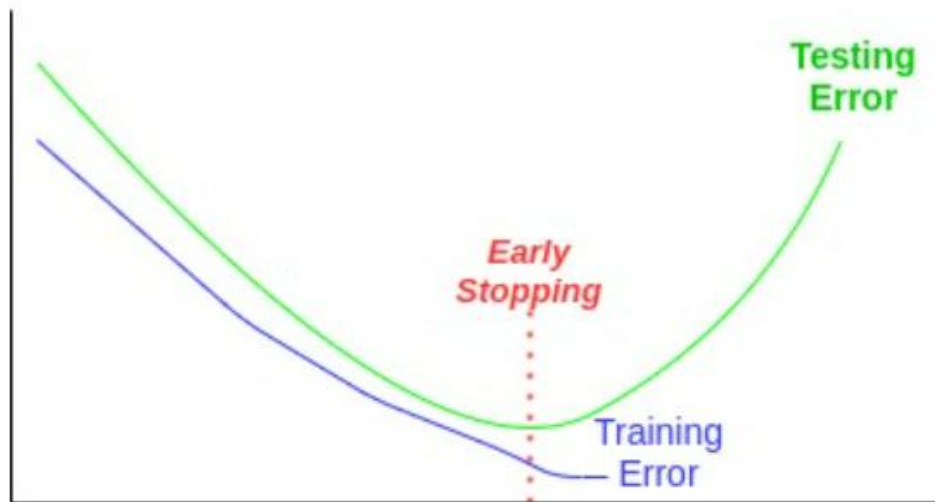
[2] Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How does batch normalization help optimization?. *Advances in neural information processing systems*, 31.

Regularization - Layer Normalization

- Layer normalization
 - Normalize each neuron (feature) in a layer according to the features in that layer – mean and std calculated for all neurons of a particular layer

Regularization

- Early Stopping



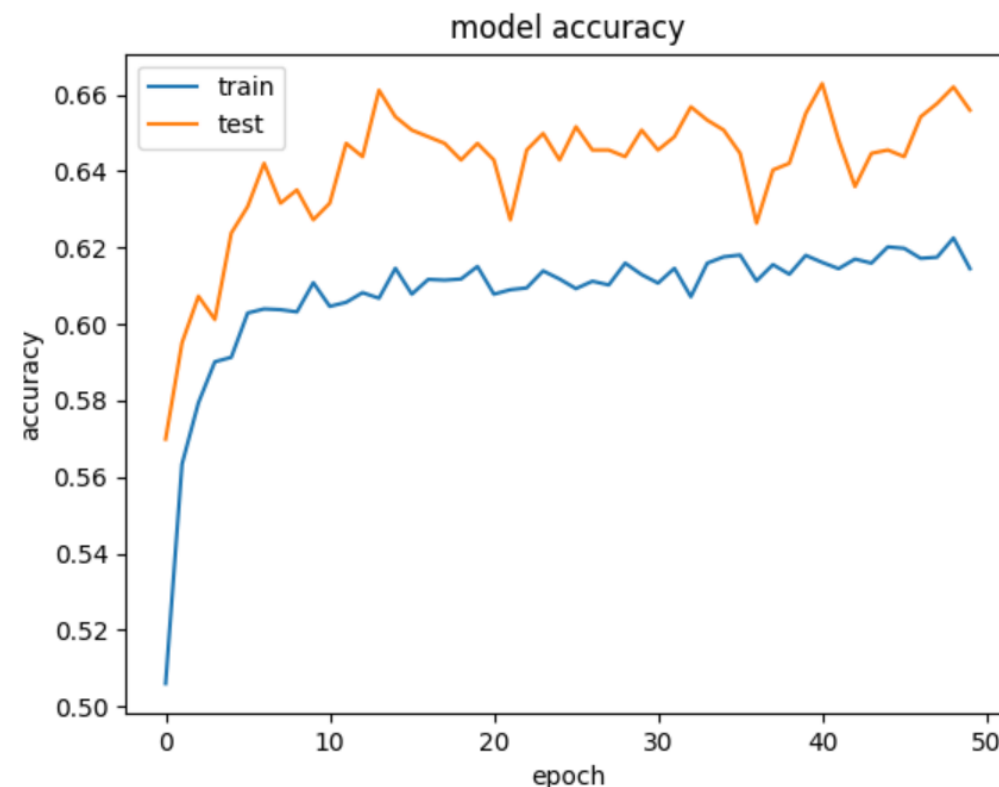
EarlyStopping

EarlyStopping class

```
keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    min_delta=0,  
    patience=0,  
    verbose=0,  
    mode="auto",  
    baseline=None,  
    restore_best_weights=False,  
    start_from_epoch=0,  
)
```

Higher Test Accuracy

- In DNN training – you may encounter scenario like test accuracy > train accuracy
- Possible reasons
 - High degree of regularization
 - Data leakage
 - Check any crossover between test and train data
 - Test data are systematically easier
 - Random
 - Bug in the code



Weight Initialization

- Weight initializers represent how the initial values of a neural network layer's weight matrix are set
 - Initial weight values in the forward pass in the first iteration
- DNN are sensitive to the initial weights
- When DNN was successfully introduced – common weight initializer was $N(0, 0.01)$
 - Was not sufficient; leads towards – vanishing gradients, dead neuron problems
- **Glorot** and **Bengio** studied the problem and proposed new initializer in 2010
 - Based on the number of input and output neurons
 - Glorot (Xavier) initializer works well for many cases and still popular in DL
- **He** showed that *Glorot Initializer* does not work well with ReLU activation
 - Extended the *Glorot Initializer* – considering only the number of input neurons and scaling by $\sqrt{2}$

Weight Initialization

- Can we initialize all the weights @ 0 or @ same number?
 - Symmetry breaking problem
- Can we initialize weights by throwing random numbers?
 - Vanishing and exploding gradient

Weight Initialization

- Can we initialize all the weights @ 0 or @ same value?
 - **Symmetry breaking problem**
- Can we initialize weights by throwing random numbers?
 - Vanishing and exploding gradient
 - Gradients might vanish/explode with the randomly thrown values;
 - During backpropagation, small gradient $\rightarrow$ smaller gradients $\rightarrow$ vanishing gradient
 - During backpropagation, large gradient $\rightarrow$ larger gradients $\rightarrow$ exploding gradient

Weight Initialization

- Can we initialize all the weights @ 0 or @ same value?
 - During backpropagation, weights are update based on previous weights; since all the weights are zero → weights update does not occur → no learning

Zeros and Ones Initialization. With all weights initialized to 0 (1), all weights are the same, and the activation in all neurons is also the same. That way, the loss function's derivative is the same for every weight in a weight matrix of a layer. When all weights have the same value, in all iterations, this makes hidden layers symmetric. Every neuron of the layer computes the same function, so the model behaves like a linear model.

Weight Initialization

- Random Initialization with a Range
 - Standard Normal distribution: $N(0,1)$
 - May lead to exploding gradient problem

If the X_i 's are independent, then $\text{Cov}(X_i, X_j) = 0$ for $i \neq j$. In this case, we can write

$$\text{If } X_1, X_2, \dots, X_n \text{ are independent, } \text{Var} \left(\sum_{i=1}^n X_i \right) = \sum_{i=1}^n \text{Var}(X_i).$$

Weight Initialization

- Xavier Initialization
 - Weights initialize using Standard Normal distribution works well
 - How we can do better:
 - Initialize weights that produce outputs following similar distribution across all neurons
 - Help convergence during training → faster and effective training
 - How Xavier initialization make sure similar distribution for all neurons?

assumptions:

- Weights and inputs are centered at zero
- Weights and inputs are independent and identically distributed
- Biases are initialized as zeros
- We use the $\tanh()$ activation function, which is approximately linear with small inputs: $Var(a^{[l]}) \approx Var(z^{[l]})$

$$W_{i,j}^{[l]} = \mathcal{N}\left(0, \frac{1}{n^{[l-1]}}\right)$$

$$Var(a_i^{[\ell-1]}) = Var(a_i^{[\ell]})$$

$$= Var(z_i^{[\ell]}) \quad \leftarrow \text{linearity of } \tanh \text{ around zero } \tanh(z) \approx z$$

$$= Var\left(\sum_{j=1}^{n^{[\ell-1]}} w_{ij}^{[\ell]} a_j^{[\ell-1]}\right)$$

$$= \sum_{j=1}^{n^{[\ell-1]}} Var(w_{ij}^{[\ell]} a_j^{[\ell-1]}) \quad \leftarrow \text{variance of independent sum } Var(X + Y) = Var(X) + Var(Y)$$

$$= \sum_{j=1}^{n^{[\ell-1]}} E[w_{ij}^{[\ell]}]^2 Var(a_j^{[\ell-1]}) + E[a_j^{[\ell-1]}]^2 Var(w_{ij}^{[\ell]}) + Var(w_{ij}^{[\ell]}) Var(a_j^{[\ell-1]}) \quad \leftarrow \text{variance of independent product } Var(XY) = E[X]^2 Var(Y) + E[Y]^2 Var(X) + Var(X) Var(Y)$$

$$= n^{[\ell-1]} Var(w_{ij}^{[\ell]}) Var(a_j^{[\ell-1]}) \implies Var(W) = \frac{1}{n^{[\ell-1]}}$$

Weight Initialization

- *He* Norm Initialization

Weight Initialization

- What is the good combination of activation and weight initialization

Effect of Batch Size!

- Effect of Batch Size!
- Modern DNN training based on mini-batch SGD
 - Large batch size increases the available computational parallelism
 - Reduce the number of parameter updates required to train a model
 - Allow distributed processing in a larger number of nodes – reduce training time
 - However, increasing the batch size the test accuracy is often fall
 - Small batch training showed improved generalization performance and optimization convergence
 - Advantage of requiring a significantly smaller memory footprint

Effect of Batch Size!

- Effect of Batch Size!



Tweet



Yann LeCun
@ylecun



Training with large minibatches is bad for your health.
More importantly, it's bad for your test error.
Friends dont let friends use minibatches larger than 32.



arxiv.org

Revisiting Small Batch Training for Deep Neural Networks
Modern deep neural network training is typically based on
mini-batch stochastic gradient optimization. While the use ...

Effect of Batch Size!

- Effect of Batch Size!
 - It is showed that – using small batch sizes achieves best training stability and generalization performance
 - The best results obtained @ batch size, $m = 32$ or smaller
 - However, batch size ($m= 32$ or 64) can be increased with BN and larger datasets

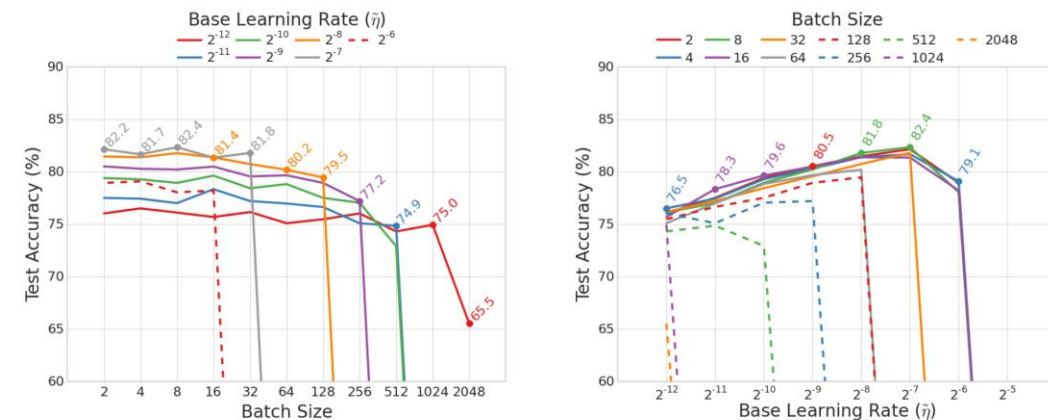


Figure 1: Test performance of reduced AlexNet model without BN, for different batch sizes and increasing values of $\tilde{\eta} = \eta/m$. CIFAR-10 dataset without data augmentation.

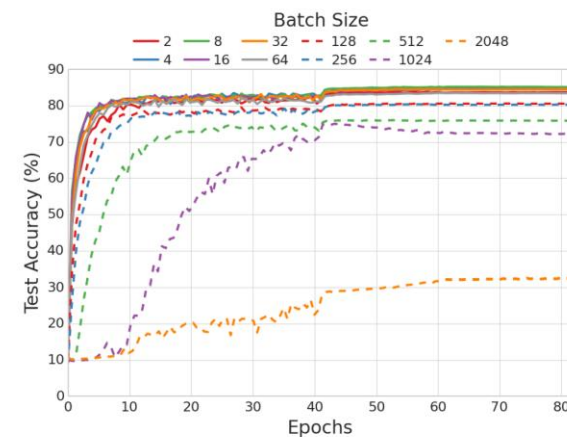


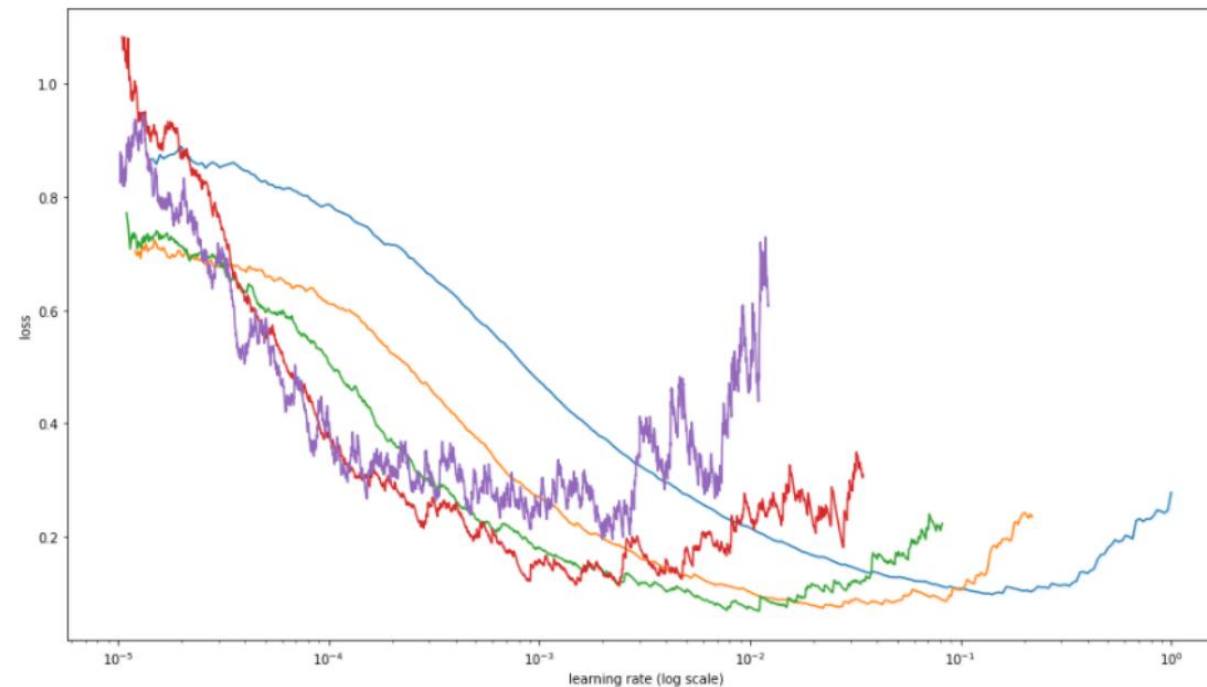
Figure 4: Convergence curves for ResNet-32 model with BN, for $\tilde{\eta} = 2^{-8}$ and for different values of the batch size. CIFAR-10 dataset without data augmentation.

Effect of Batch Size!

- Effect of Batch Size!
 - If we increase batch size than also increase the learning rate
 - General Rule- batch size increase by $k \rightarrow$ learning rate increase by $\sqrt{k}$

LOSS vs. LEARNING RATE FOR DIFFERENT BATCH SIZES

BATCH SIZE:



There you have it, **the relationship between learning rate error plotted using batches from 64 to 4** for the “cats vs. dogs” dataset.

Dr. Masum

Measuring Model Performance

Individual Number	1	2	3	4	5	6	7	8	9	10	11	12
Actual Classification	1	1	1	1	1	1	1	1	0	0	0	0
Predicted Classification	0	0	1	1	1	1	1	1	1	0	0	0
Result	FN	FN	TP	TP	TP	TP	TP	TP	FP	TN	TN	TN

		Predicted condition	
Total population = P + N		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

		Predicted condition	
Total 8 + 4 = 12		Cancer 7	Non-cancer 5
Actual condition	Cancer 8	6	2
	Non-cancer 4	1	3

Measuring Model Performance

$$Accuracy = \frac{TP + TN}{P + N}$$

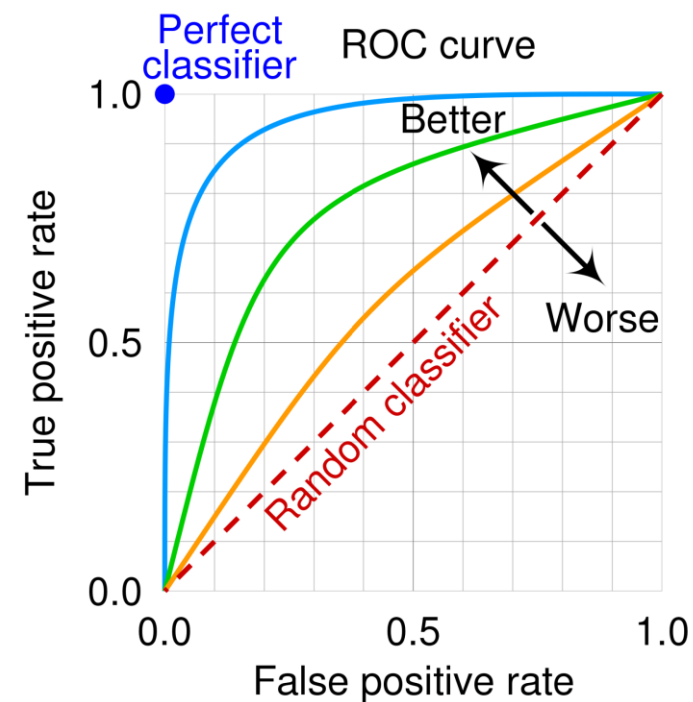
$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F1\ score = \frac{2 * Precision * Recall}{Precision + Recall}$$

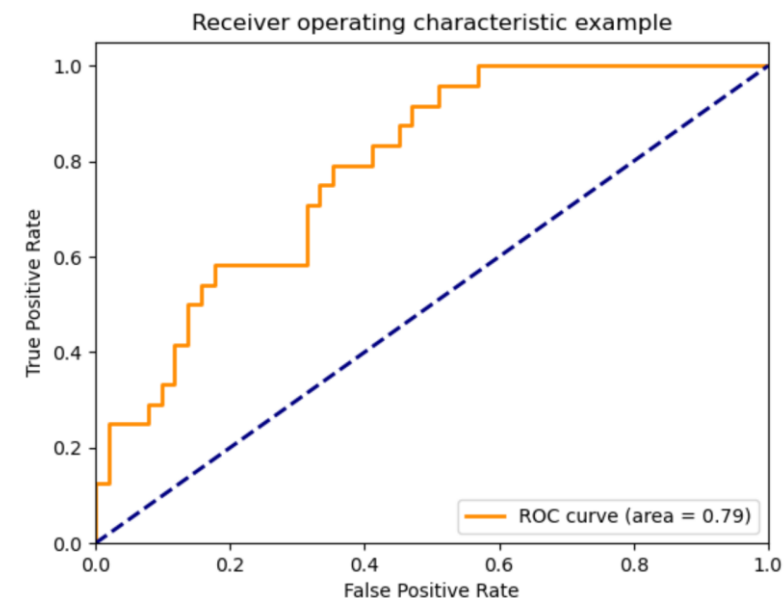
Measuring Model Performance

- ROC curve
 - A graphical plot to diagnose model performance using discrimination threshold
 - TPR vs FPR plot
 - The perfect classification point is (0,1) in the ROC space
 - 100% sensitivity $\rightarrow$ no false negatives and 100% specificity $\rightarrow$ no false positive



Measuring Model Performance

- AUC-ROC curve
 - A single value that we can easily compare to other models
 - A perfect classifier AUC score = 1
 - Area under curve can be approximated using calculus formulas, e.g., Riemann Sum, Trapezoidal Rule



Measuring Model Performance

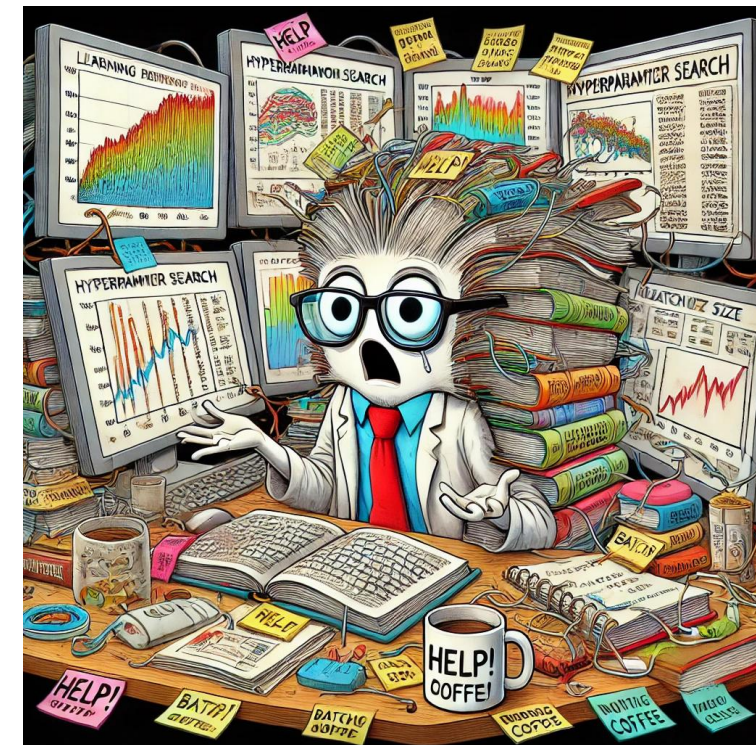
- General Rule – size of training data should be 5 to 10 times the size of weights of the network
- Other Rule: $N = \frac{|W|}{1-a}$
 - Where, a is the expected accuracy and W is weights

- **Hyperparameter Tuning** – Adjusting the settings of a ML or DL optimize its performance
 - Example: Learning rate, batch size, number of layers, dropout rate, optimizer type.
 - **Trainable Parameters:** Weights and biases, optimized during training.
 - **Hyperparameters:** Set before training and affect the model's behavior.
- Advantages – Improve models performance, prevents overfitting, optimizing training time

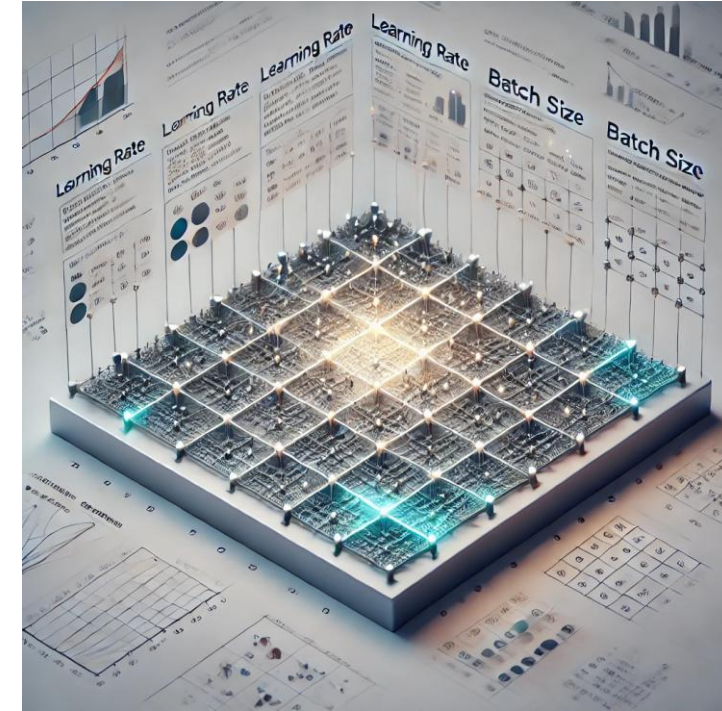


Types of Hyperparameter Tuning

- **Manual Search:** Trial-and-error based; suitable for small models or experienced practitioners.
- **Grid Search:** Exhaustive search over a specified range of hyperparameters.
- **Random Search:** Randomly samples hyperparameter combinations within defined bounds.
- **Bayesian Optimization:** Builds a probabilistic model to efficiently explore the hyperparameter space.
- **Advanced Methods:**
 - Population-Based Training (PBT), Hyperband, and Neural Architecture Search (NAS).



- **Grid Search** – Exhaustive Exploration : Explores all possible combinations of hyperparameters in a predefined grid.
- **Pros:**
 - Simple and systematic.
 - Guarantees to find the best combination (if within the grid).
- **Tools:** GridSearchCV in Scikit-learn.
- **Example:** Learning rates: [0.01, 0.1, 0.001], Batch sizes: [16, 32, 64].
 - **Total Combinations?**



Disadvantages of Grid Search:

- **Exponential Growth in High Dimensions:** computationally infeasible for high-dimensional search spaces.
- **Inefficiency for Real-Valued Hyperparameters:** Grid search often misses optimal values due to coarse discretization of continuous hyperparameters.
- **Uniform Sampling Ignores Parameter Sensitivities:** It treats all hyperparameters equally
- **Lack of Adaptivity:** Grid search does not use intermediate results to refine the search, leading to redundant computations.
- **Inflexibility for Complex Search Spaces:** It assumes a regular search space, failing to adapt to non-linear or irregular parameter landscapes.

Random Search – Samples hyperparameter combinations randomly from the search space.

- Computationally efficient, but no guarantees to find the optimal combination & may still require large samples for high-dimensional spaces.
- **Tools:** RandomizedSearchCV in Scikit-learn.
- Often performs as well or better than Grid Search with fewer iterations.

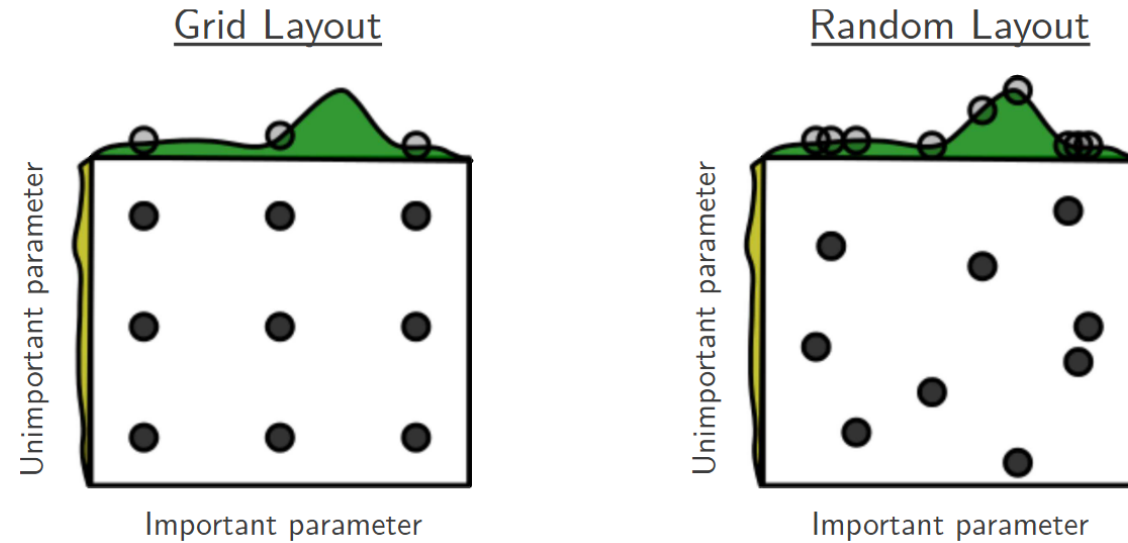


Figure 1: Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

- **Bayesian Optimization** – Builds a probabilistic model (e.g., Gaussian Process) of the objective function and uses acquisition functions (e.g., Expected Improvement) to decide the next hyperparameter set to evaluate.
 - Efficient in exploring complex search spaces.
 - Bayesian optimization can have overhead due to surrogate model training and acquisition function optimization but requires fewer evaluations overall
- **Tools:** Optuna, Ax, and GPyOpt.

Bayesian Optimization Process:

- Randomly evaluate a few hyperparameter combinations
- Use the initial results to build a simple **surrogate model** that estimates how different hyperparameters affect performance.
- Optimize the **Acquisition Function**: Use the surrogate model to choose the next set of hyperparameters to try
- Evaluate New Hyperparameters: Train the actual model with the suggested hyperparameters and record the performance metrics.
- Update Surrogate Model: Add the new trial's results to the dataset and retrain the surrogate model for better predictions.
- Repeat process
- Finalize best hyperparameters

Criteria	Grid Search	Random Search	Bayesian Optimization
Exhaustiveness	High	Medium	Medium
Efficiency	Low	Medium	High
Computational Cost	High	Medium	Medium
Scalability	Low	High	Medium
Guarantee of Best Result	Yes (if grid is sufficient)	No	No (but more likely)

